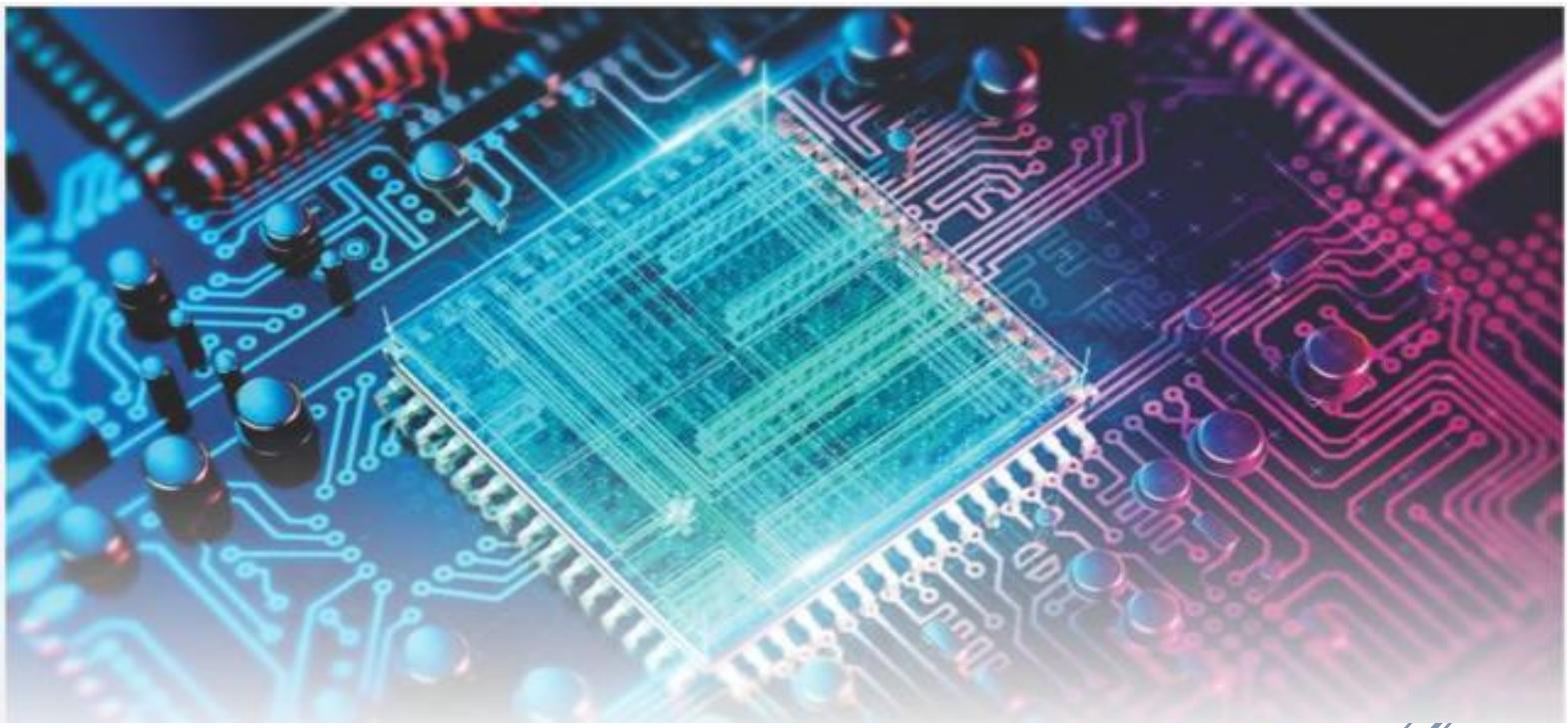


(DEEMED TO BE UNIVERSITY)

EMBEDDED SYSTEMS PROGRAMMING AND RTOS FOUNDATIONS

24IN2223HF

STUDENT ID:

STUDENT NAME:

ACADEMIC YEAR: 2025-26

Table of Contents

1. **Session 01: Introductory Session**
2. **Session 02: Board Bring-Up and Blinky**
3. **Session 03: GPIO Input/Output Programming**
4. **Session 04: System Clock and Power Modes**
5. **Session 05: Register Manipulation and Bitwise Operations**
6. **Session 06: Digital Thermometer with LCD**
7. **Session 07: PWM Generation and Servo/Motor Control**
8. **Session 08: Interrupts and Event-Driven Programming**
9. **Session 09: ADC Data Acquisition and Plotting**
10. **Session 10: UART Serial Communication**
11. **Session 11: RTOS Task Scheduling**
12. **Session 12: Inter-Task Communication (Queues/Semaphores)**
13. **Session 13: Timer-Based Real-Time Applications**
14. **Session 14: Project**

A.Y. 2025-26 LAB/SKILL CONTINUOUS EVALUATION

S.No	Date	Experiment Name	Pre-Lab (10M)	In-Lab (25M)			Post-Lab (10M)	Viva Voce (5M)	Total (50M)	Faculty Signature
				Program/ Procedure (5M)	Data and Results (10M)	Analysis & Inference (10M)				
1.		Introductory Session	-NA-							
2.		Board Bring-Up and Blinky								
3.		GPIO Input/Output Programming								
4.		System Clock and Power Modes								
5.		Register Manipulation and Bitwise Operations								
6.		Digital Thermometer with LCD								
7.		PWM Generation and Servo/Motor Control								
8.		Interrupts and Event-Driven Programming								
9.		ADC Data Acquisition and plotting								
10.		UART Serial Communication								
11.		RTOS Task Scheduling								
12		Inter-Task Communication (Queues/Semaphores)								

S.No	Date	Experiment Name	Pre-Lab (10M)	In-Lab (25M)			Post-Lab (10M)	Viva Voce (5M)	Total (50M)	Faculty Signature
				Program/ Procedure (5M)	Data and Results (10M)	Analysis & Inference (10M)				
13.		Timer-Based Real-Time Applications								
14.		Project								

Experiment Title 1: Board Bring-Up and Blinky

Aim/Objective:
To Check the blinking of on-Board LED

Components Required: STM32 NUCLEOC0316C BOARD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

The experiment aims in The Nucleo-C031C6 is a development board based on the STM32C031C6 microcontroller, which belongs to the STM32C0 series of ARM Cortex-M0+ devices.
The board includes:

- **The MCU (STM32C031C6)**
- **An onboard ST-LINK/V2-1 debugger**
- **A user LED (LD3) electrically connected to GPIO pin PA5**
- **A USB connector for power and programming**

The board setup process consists of preparing the microcontroller so it can execute user-defined instructions—mainly by configuring clocks, enabling peripherals, and defining GPIO behaviour

Pre-Lab:

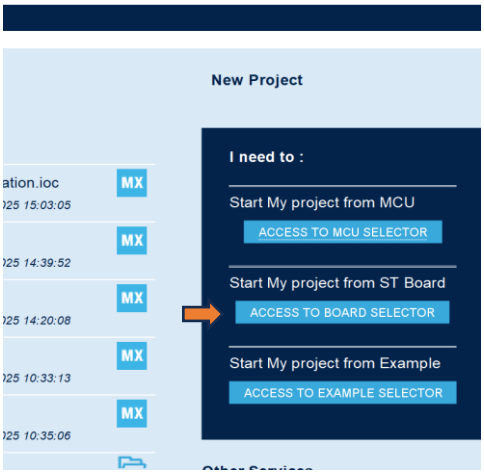
1. **Explain the features of STM32 NUCLEOC031C6?**

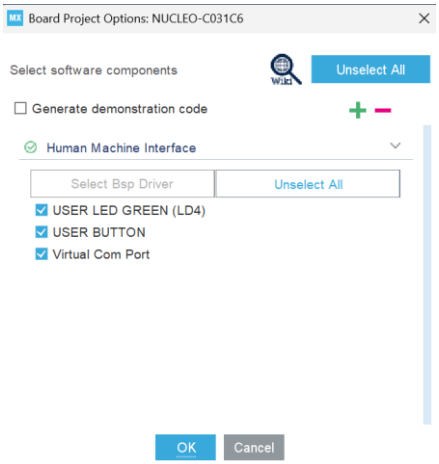
2. **What are uses of an on board LED? Explain its significance ?**

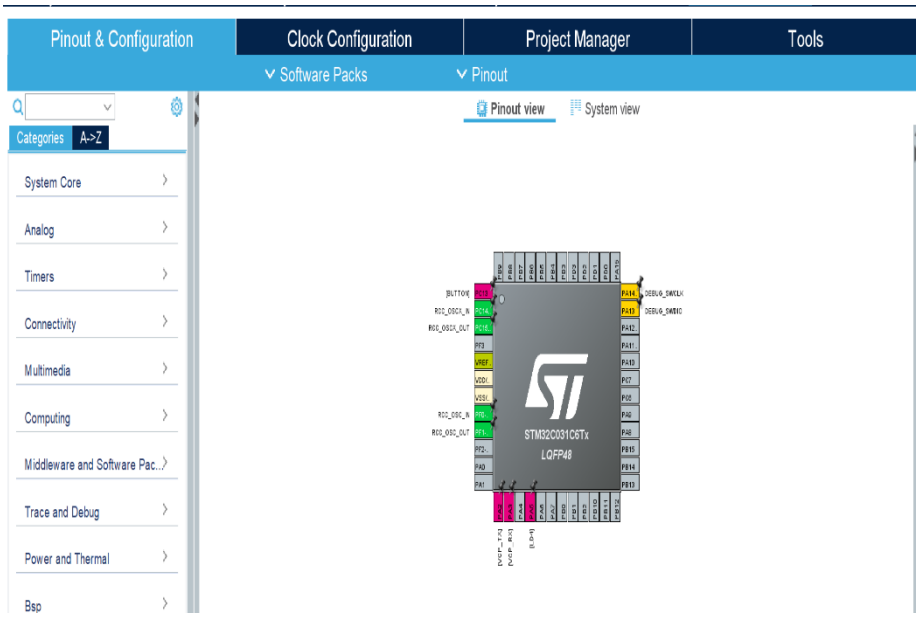
In-Lab:

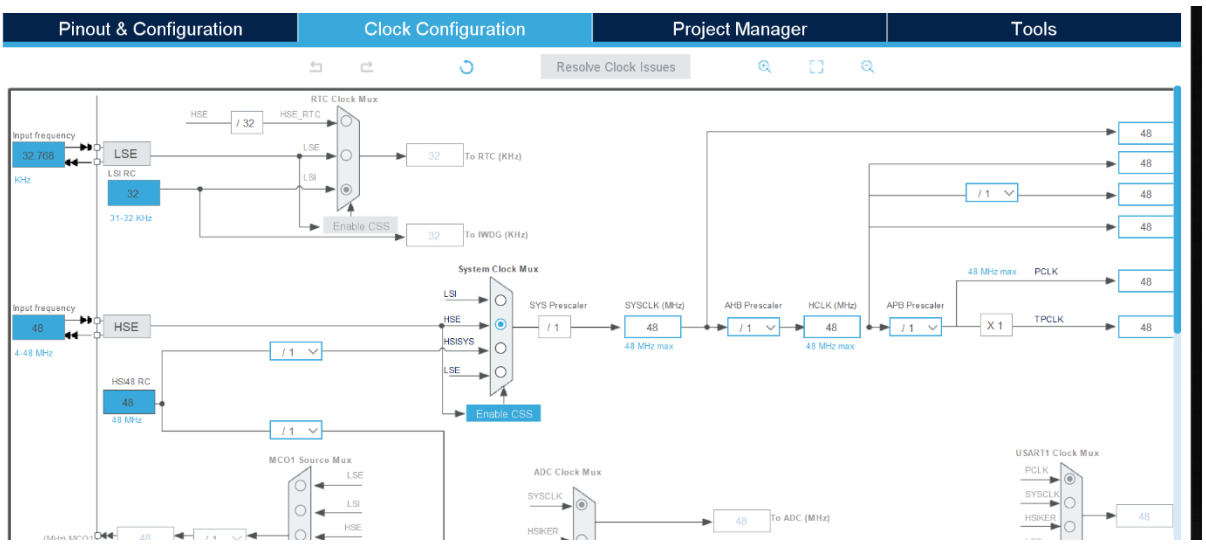
Procedure:

1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**
5. **OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE**

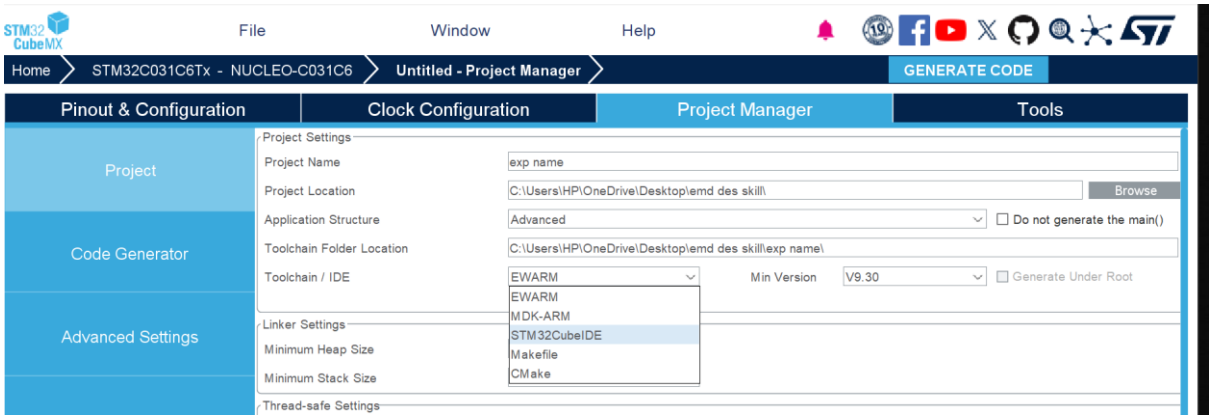








- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **In main .c file write the program needed for blinking the LED.**

Output:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions

1. **Explain the different types of clocks in NUCLEOC031C6?**

2. **State some practical uses of ON BOARD LED ?**

3. **Write the steps involved in your experiment and your understanding on blinking of on board led ?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 2 : GPIO Input/Output Programming

Aim/Objective:
Blinking of an On-board LED using an on-board push button

Components Required: STM32 NUCLEOC0316C BOARD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

The experiment aims in The Nucleo-C031C6 is a development board based on the STM32C031C6 microcontroller, which belongs to the STM32C0 series of ARM Cortex-M0+ devices.
The board includes:

- **The MCU (STM32C031C6)**
- **An onboard ST-LINK/V2-1 debugger**
- **A user LED (LD3) electrically connected to GPIO pin PA5**
- **A USB connector for power and programming**

The board setup process consists of preparing the microcontroller so it can execute user-defined instructions—mainly by configuring clocks, enabling peripherals, and defining GPIO behaviour

Pre-Lab:

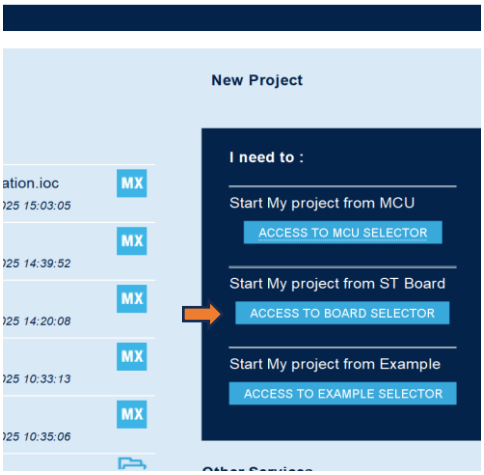
1. **Explain the features of STM32 NUCLEOC031C6?**

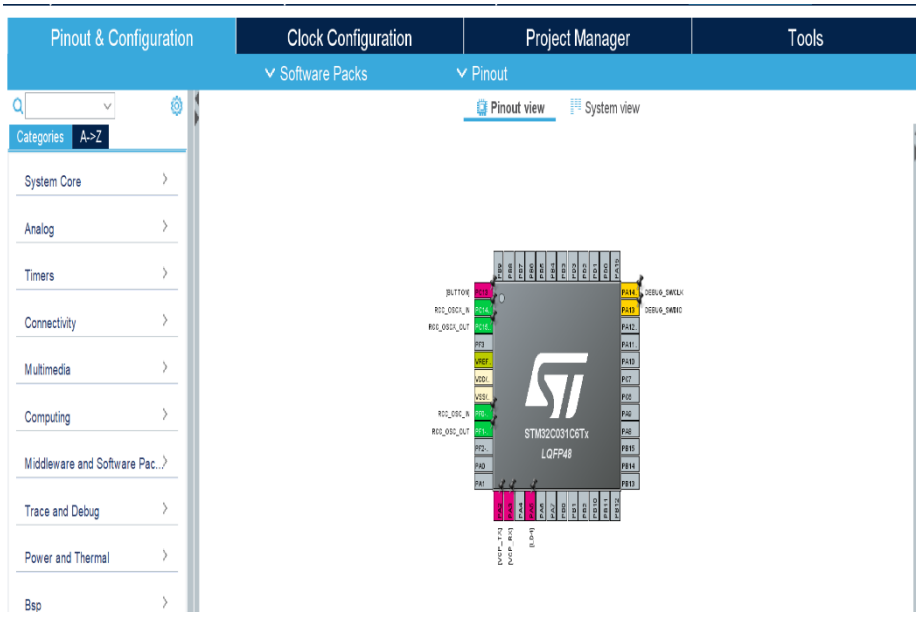
2. **What are uses of a Push Button? Explain it's significance ?**

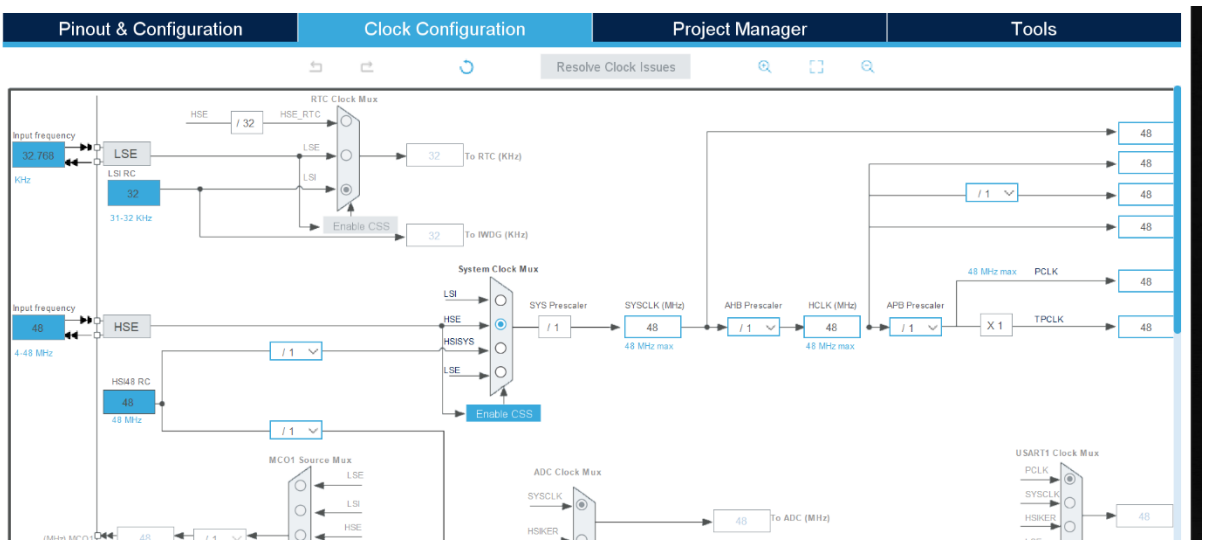
In-Lab: To blink an on board LED using on board PUSHBUTTON configured at PC13 pin lock in figure given

Procedure:

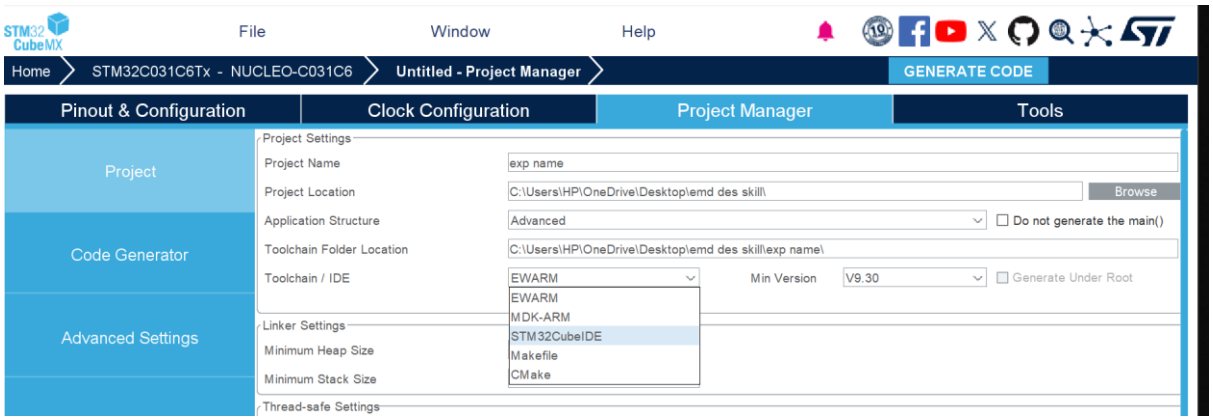
1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**
5. **OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE**







- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **After the successful generation of code just open your main.c file and check for the while loop inside the while loop paste or write the following code to toggle led using pushbutton.**

Output:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **Explain the uses of PushButton?**

2. **HOW GPIO INPUT/OUTPUT configuration Is done in NUCLEOC031C6?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 3 : System Clock and Power Modes

Aim/Objective:
Study on System Clock and Power Modes

Components Required: STM32 NUCLEOC0316C BOARD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

The experiment aims in The Nucleo-C031C6 is a development board based on the STM32C031C6 microcontroller, which belongs to the STM32C0 series of ARM Cortex-M0+ devices.
The board includes:

- **The MCU (STM32C031C6)**
- **An onboard ST-LINK/V2-1 debugger**
- **A USB connector for power and programming**

The board setup process consists of preparing the microcontroller so it can execute user-defined instructions—mainly by configuring clocks, enabling peripherals, and defining GPIO behaviour

Pre-Lab:

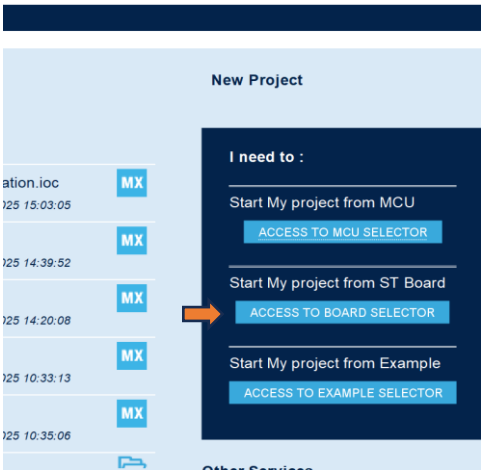
1. **Explain the features of STM32 NUCLEOC031C6?**

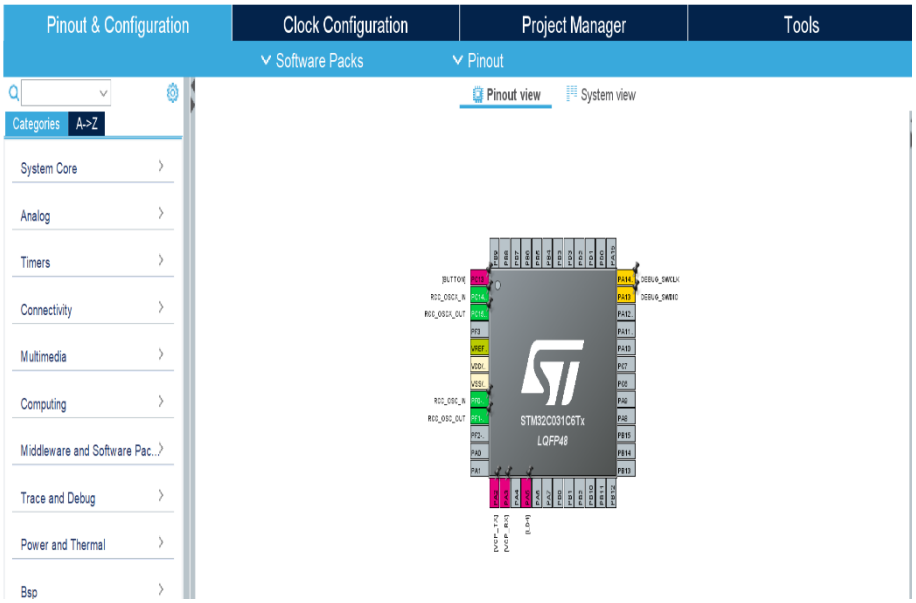
2. **What is a System clock ? Explain it's significance ?**

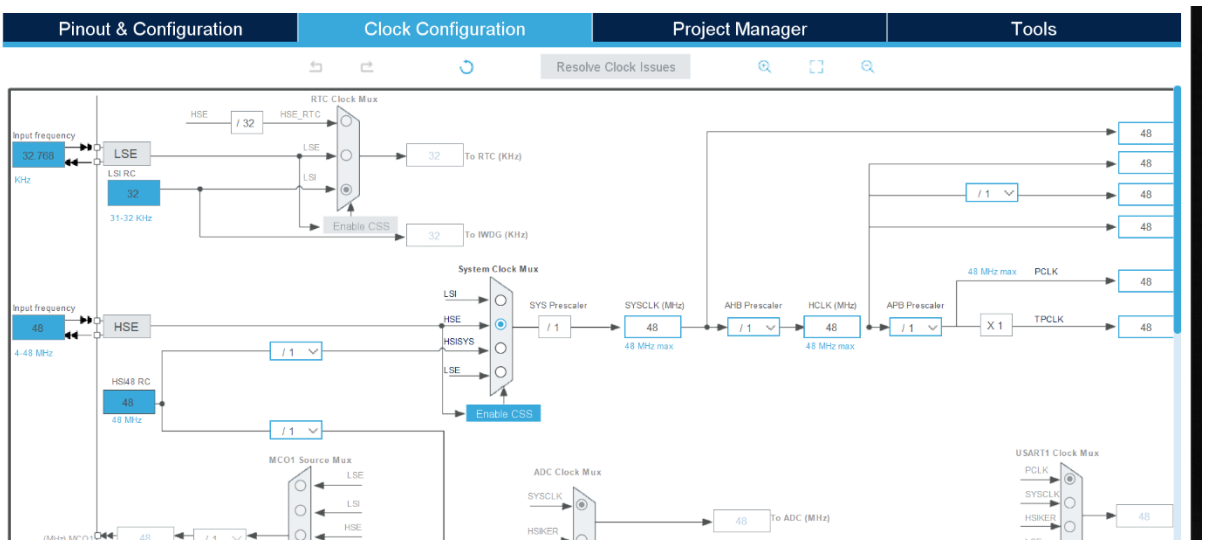
In-Lab: System Clock and Power Modes

Procedure:

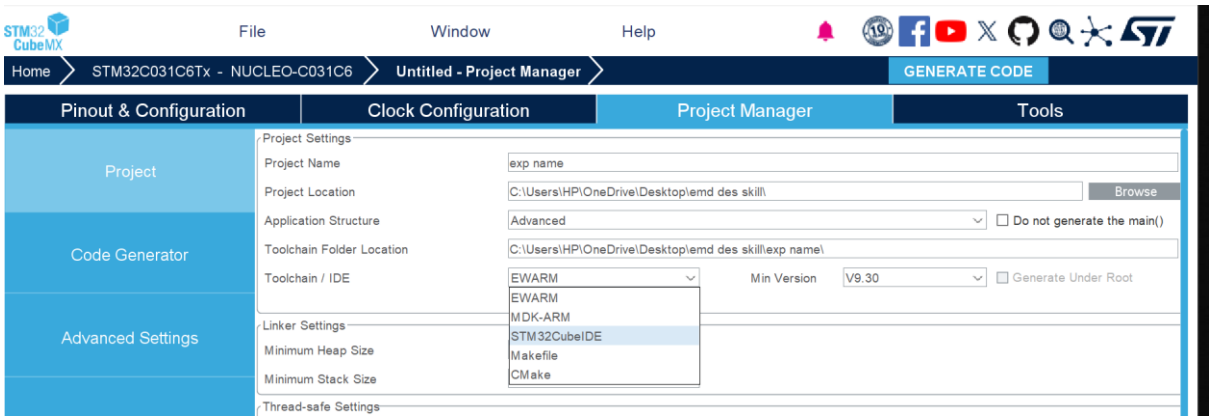
1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**
5. **OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE**







- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **After the successful generation of code just open your main.c file and check for the while loop inside the while loop paste or write the following code to toggle led using pushbutton.**

Output

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **Explain the uses of System clock? what are different power modes?**

2. **What is the default clock frequency of nucleoc0316c ?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 4 : Register Manipulation and Bitwise Operations

Aim/Objective:
Register Manipulation and Bitwise Operations

Components Required: STM32 NUCLEOC0316C BOARD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

Register manipulation is the technique of controlling microcontroller peripherals by writing directly to hardware registers, which are special memory-mapped addresses.

On STM32C031, every peripheral (GPIO, TIM, ADC, RCC, etc.) is controlled by registers, e.g.:

- **RCC->IOPENR – enables GPIO port clocks**
- **GPIOA->MODER – configures GPIO pin mode**
- **GPIOA->ODR – outputs digital values**
- **GPIOA->IDR – reads input values**

The STM32 HAL library provides high-level functions, but register-level programming gives:

- **Faster execution**
- **More precise control**
- **Better learning of how the MCU works internally**

Operator	Meaning	Example
&	AND	Clear bits
`	`	OR
^	XOR	Toggle bits
~	NOT	Invert bits
<<	Left shift	Shift bits left
>>	Right shift	Shift bits right

Pre-Lab:

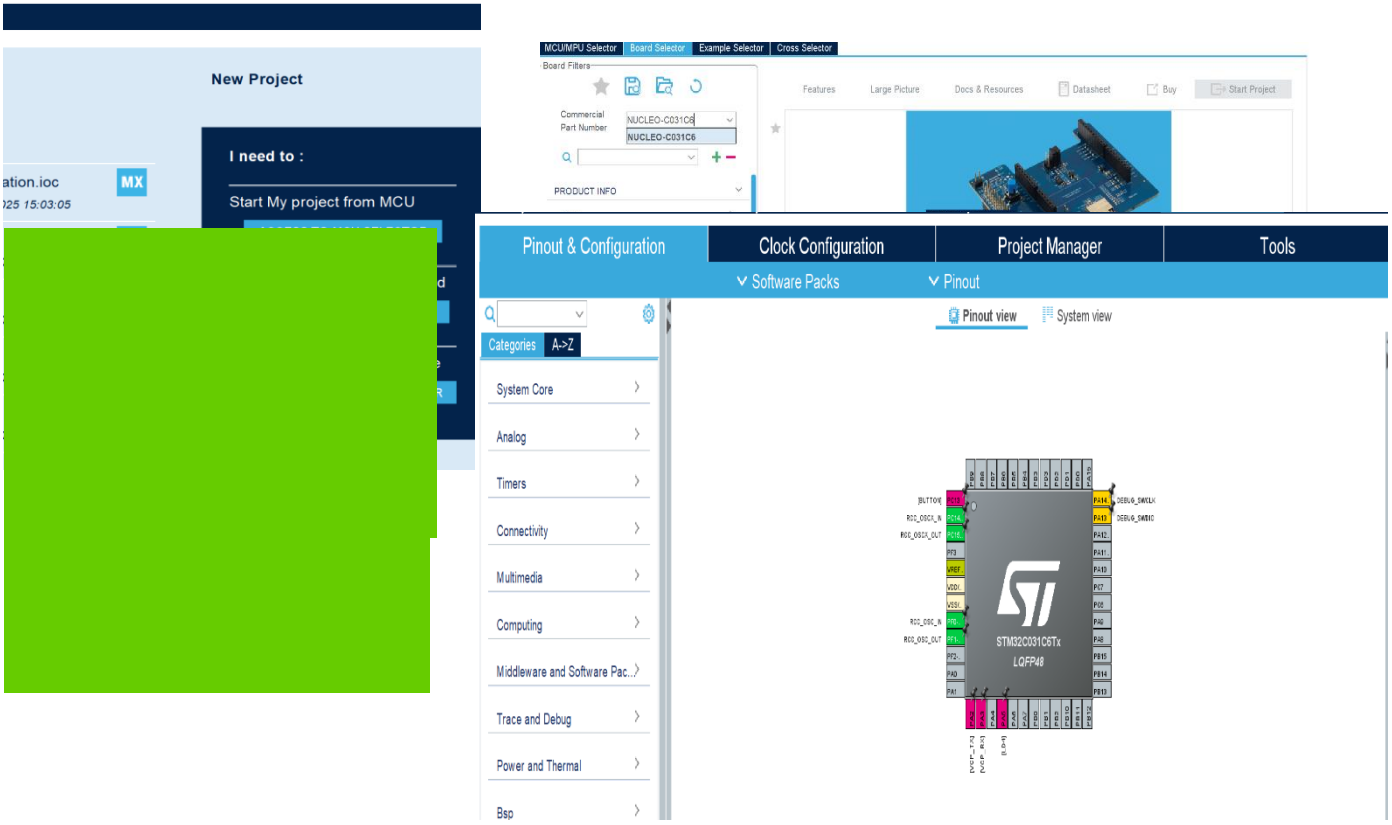
1. **Explain the types of register in nucleoc031c6?**

2. **What are bitwise operators and their uses?**

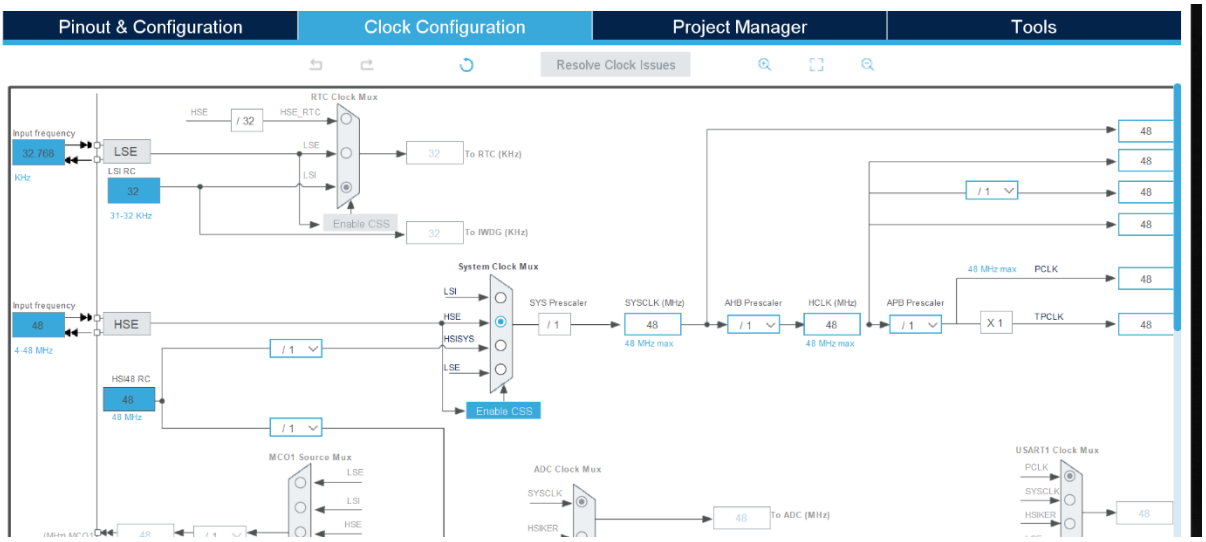
In-Lab: Register Manipulation and Bitwise Operation

Procedure:

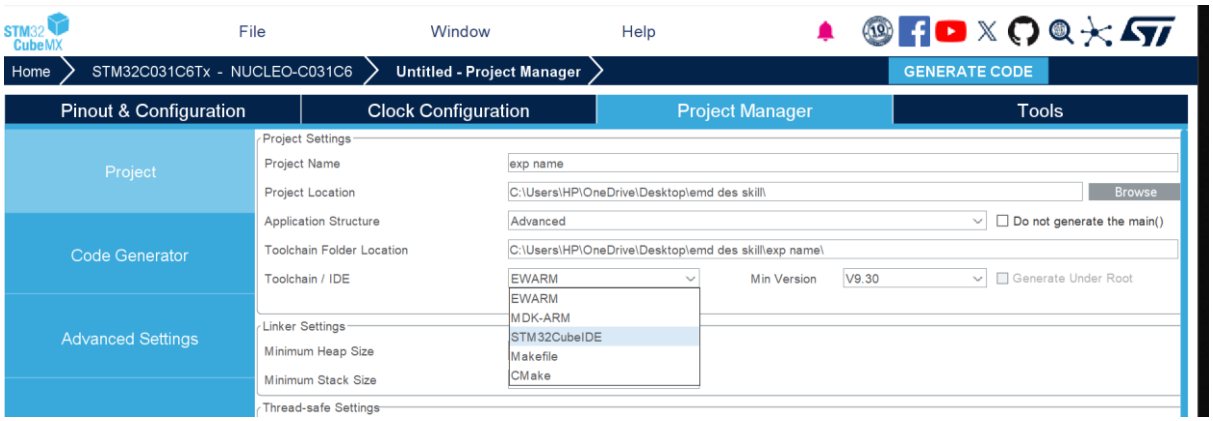
1. **Open STM32CubeIDE.**
2. **Create New STM32 Project.**
3. **Select Nucleo-C031C6 board.**
4. **Name project and select Empty / No HAL or HAL (user will override).**



5. **Generate project.**
6. **Open main.c.**



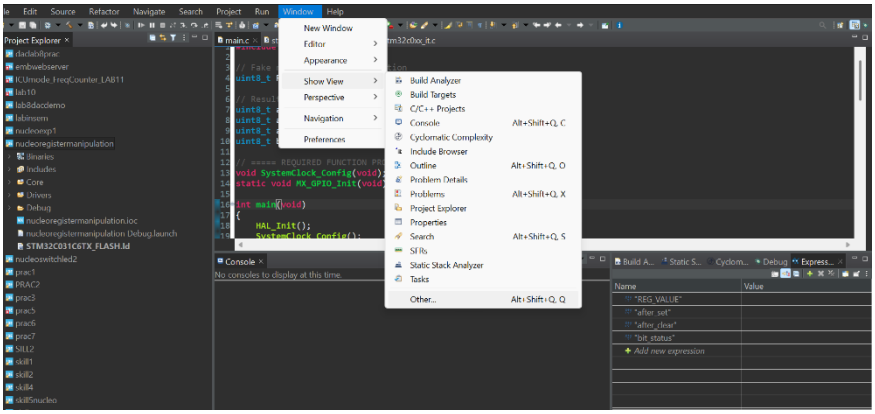
- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**

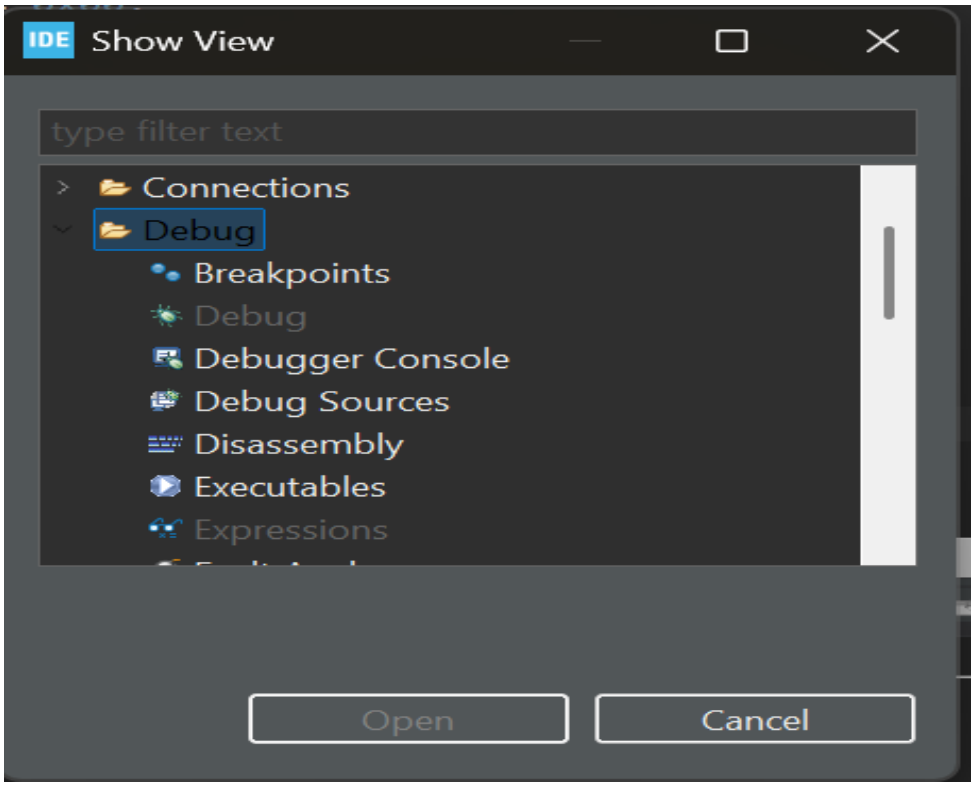


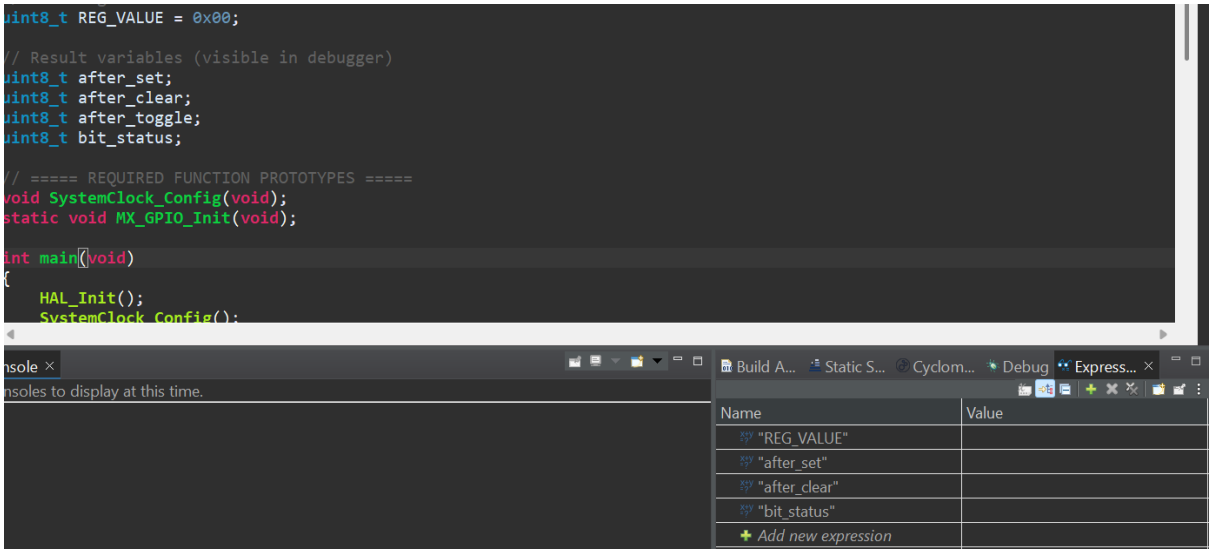
PROCEDURE IN CUBE IDE:

1. **Open cube ide when project is opened after clicking on generate code**
2. **After writing the code, Run -> debug**
3. **After successful debug click on**
   Windows -> show view-> other-> Debug->Expressions
4. **You get an expression window table on side pane.**
5. **Click on plus symbol or add symbol and add the values REG VAL , AFTER SET, AFTER TOGGLE, AFTER RESET, AFTER MANIPULATION ETC VALUES**

6. **RUN THE CODE DEBUG AGAIN CLICK ON PLAY/CONTINUE SYSMBOL AND OBSERVE THE MANIPULATIO OF BITS**







PASTE THE VALUES THAT ARE IN PROGRAM WHILE ADDING EXPRESSIONS

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

Experiment	<TO BE FILLED BY STUDENT>	**Student ID**	<TO BE FILLED BY STUDENT>
Date	<TO BE FILLED BY STUDENT>	**Student Name**	<TO BE FILLED BY STUDENT>

VIVA-VOCE Questions:

1. **DEFINE A REGISTER AND WHAT ARE BASIC TYPES OF REGISTERS USED ?**

2. **WITH AN EXAMPLE TAKE AN EXPRESSION AND PERFORM THE BITWISE OPERATIONS?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 5 : Digital Thermometer with LCD

Aim/Objective:
Digital Thermometer with LCD

Components Required: STM32 NUCLEOC0316C BOARD,16x2 LCD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**
- **16 X 2 LCD**
- **TEMAPRATURE LM32 OR DHT11 SENSOR**

Description:

A digital thermometer using an STM32 Nucleo board and an LCD display is an embedded system designed to measure temperature and display it digitally in real-time. This project combines a temperature sensor, a microcontroller, and an LCD module to create a compact and accurate temperature monitoring device.

Components:

1. **STM32 Nucleo Board – Acts as the microcontroller unit (MCU) to process temperature readings and control the display.**
2. **Temperature Sensor – Common options include:**
   - **LM35: Analog temperature sensor with output proportional to temperature.**
   - **DS18B20: Digital temperature sensor using the 1-Wire protocol.**
3. **LCD Display – Typically a 16x2 character LCD or a graphical LCD, used to show the measured temperature.**
4. **Connecting Wires & Breadboard – For circuit connections.**

Working Principle:

1. **Temperature Sensing:**
   The sensor detects the ambient temperature and converts it into an electrical signal (analog voltage for LM35 or digital data for DS18B20).
2. **Signal Processing:**
   The Nucleo board reads the sensor output:
   - **For analog sensors, the ADC (Analog-to-Digital Converter) on the STM32 converts voltage into a digital value.**

- **For digital sensors, data is read using a communication protocol like 1-Wire or I2C.**

3. **Temperature Calculation:**
   The MCU calculates the temperature in °C or °F using the sensor's characteristics. For example, LM35 outputs 10 mV per °C, so the MCU converts the ADC value to temperature.

4. **Display Output:**
   The calculated temperature is sent to the LCD. The display is updated periodically (e.g., every second) to show real-time temperature readings.

Pre-Lab:

1. **Explain the uses of an LCD DISPLAY?**

2. **DESCRIBE PINS OF LCD 16X2?**

In-Lab: System Clock and Power Modes

Procedure:

1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**

CONNECTIONS

LCD Pin Nucleo C031C6

SDA D15 → PB9

LCD Pin Nucleo C031C6

SCL D14 → PB8

VCC 5V

GND GND

- **DHT11 VCC → 5V**
- **DHT11 GND → GND**
- **DHT11 DATA → A1 (PA1) with pull-up 4.7kΩ to VCC**

Step 1: Create New Project

1. **Open STM32CubeMX.**
2. **Click New Project → Select STM32C031C6 → Start Project.**
3. **Give a name (e.g., DHT11_UART).**

Step 2: Configure GPIO for DHT11

1. **Go to Pinout & Configuration.**
2. **Locate PA1 (we will use this for DHT11 DATA).**
3. **Click on PA1 → Set GPIO_Input.**
4. **In GPIO Settings:**
   - **Mode: Input**

Go to Clock Configuration tab.

Make sure HSI is selected as System Clock Source.

Set SYSCLK = 48 MHz.

APB1 = 48 MHz (default for C031C6).

Click Apply.

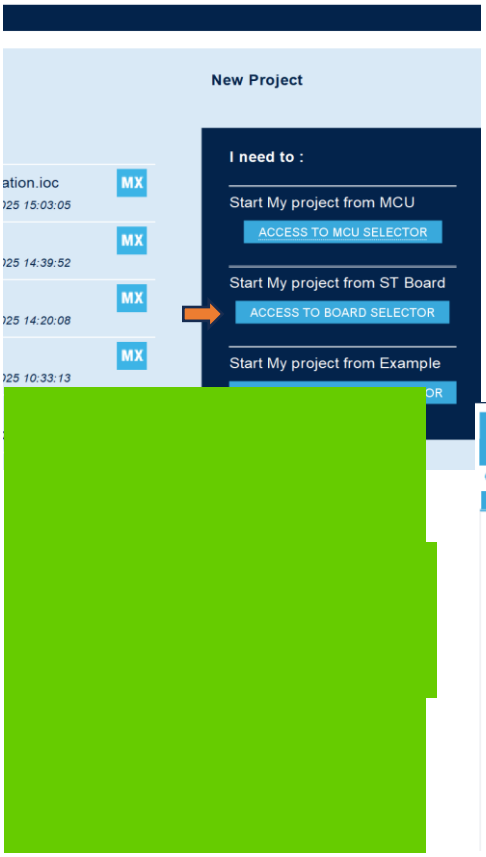
Step 3: Configure TIM3 for Microsecond Delay

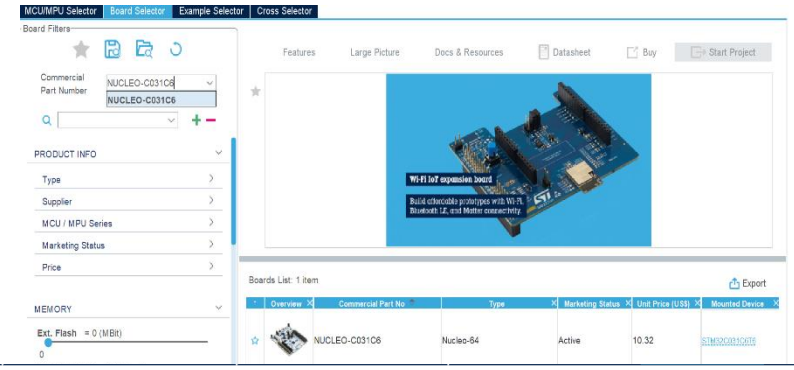
1. **Go to Timers → TIM3.**

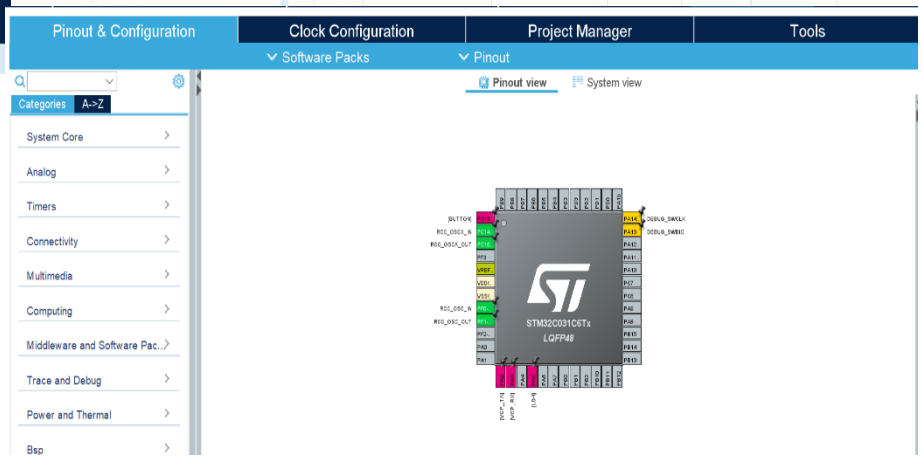
2. **Mode: Base.**
3. **Prescaler: 48-1 → this gives 1 MHz counter (1 tick = 1 µs if clock = 48 MHz).**
4. **Counter Mode: Up.**
5. **Period: 65535.**
6. **Clock Division: DIV1.**
7. **Click Apply.**

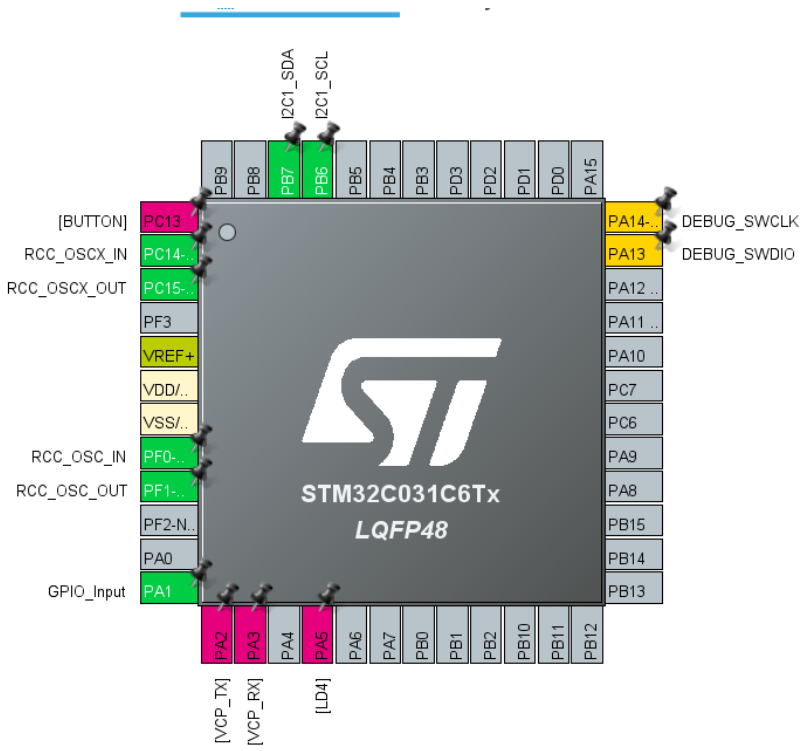
- **Go to Connectivity → USART1.**
- **Mode: Asynchronous.**

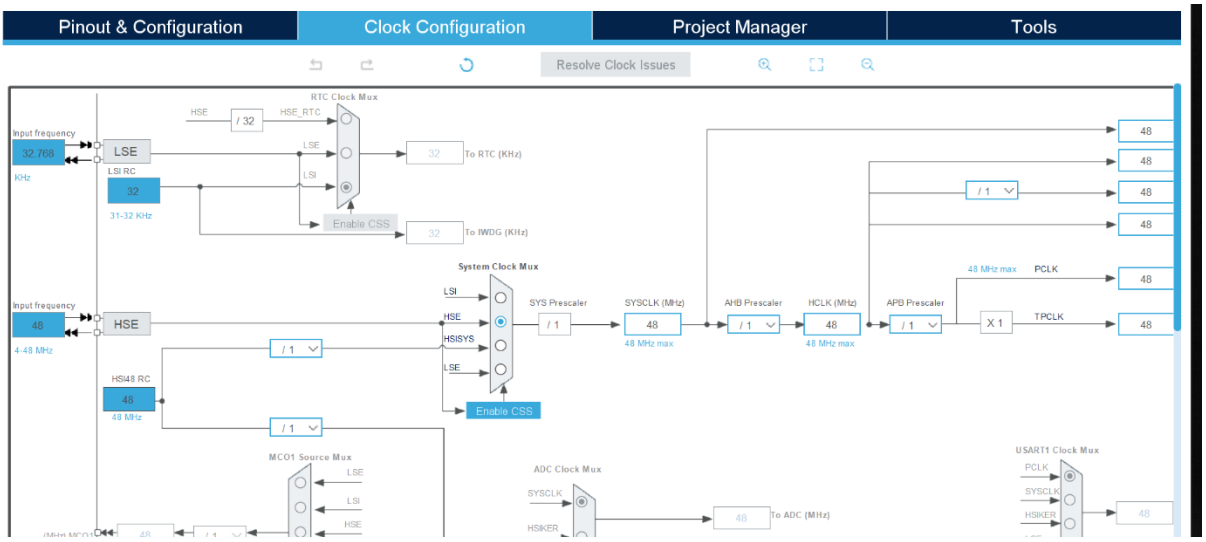
5. **OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE**



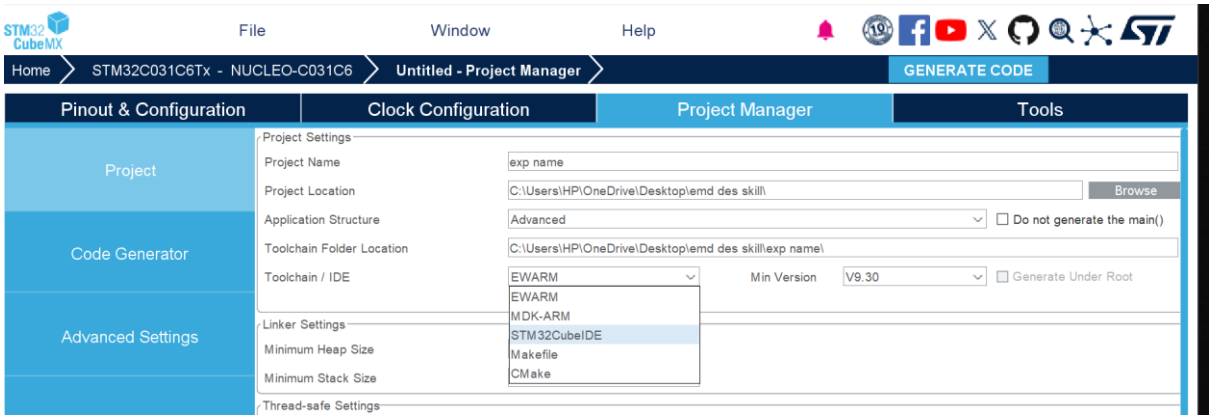








- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **Select file->Core->Src->main and copy the given program and paste**

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **Explain the uses of LCD ?**

2. **What is the default clock frequency of nucleoc0316c ?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 6: PWM Generation and Servo/Motor Control

Aim/Objective:
PWM Generation and Servo/Motor Control

Components Required: STM32 NUCLEOC0316C BOARD, SERVO MOTOR

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**
- **SERVO MOTOR**

Description:

A servo motor is a rotary actuator that allows precise control of angular position, velocity, and acceleration. Unlike a standard DC motor that spins freely, a servo motor can rotate to a specific angle within a limited range, typically 0° to 180° for hobby servos.

Pre-Lab:

1. **Explain the features of STM32 NUCLEOC031C6?**

2. **What applications of SERVO MOTOR AND WRITE ITS WIRE CONFIGURATIONS ?**

In-Lab:

Procedure:

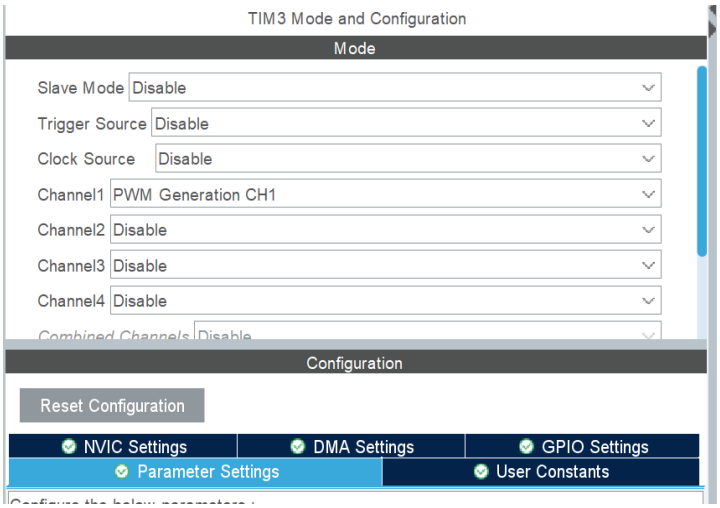
1. **Select Nucleo C031C6 as your board.**
2. **Click Start Project.**

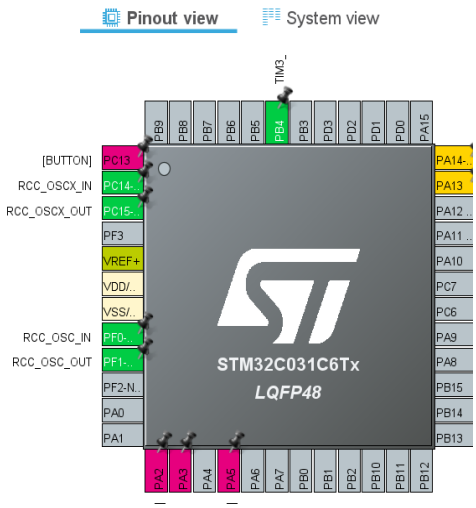
Step 2: Configure Timer for PWM on D5

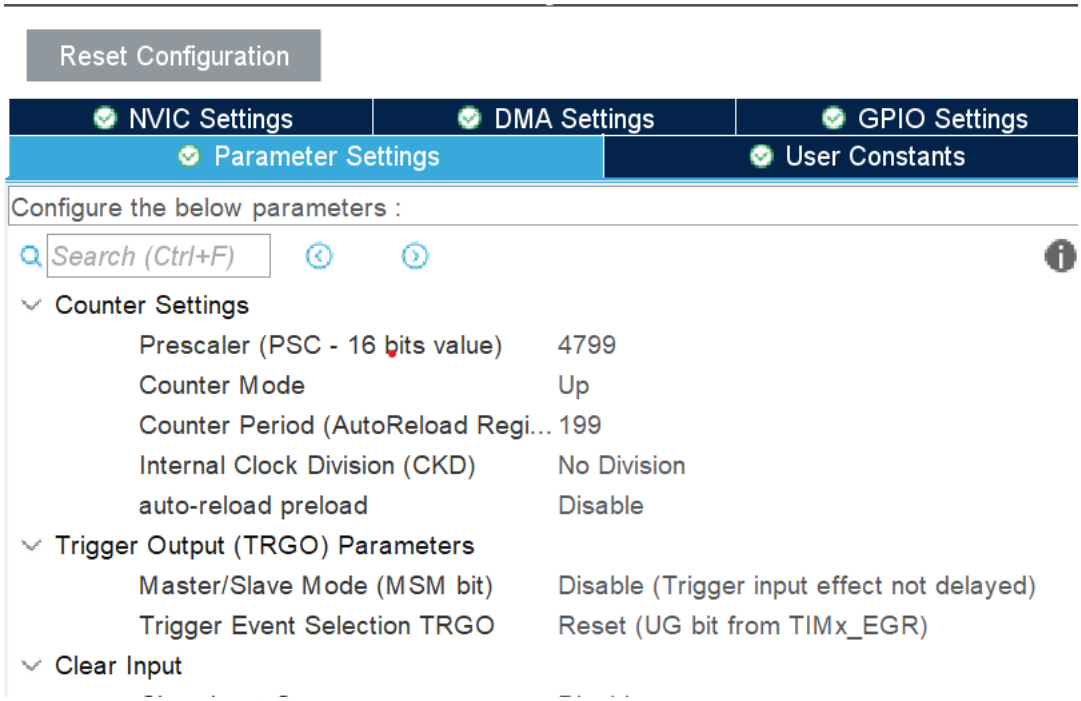
1. **Go to Pinout & Configuration tab.**
2. **Click PB4 (Arduino D5) → set it as TIM3_CH1.**
3. **In the Peripherals pane → enable TIM3.**
4. **Click TIM3 → Mode → select PWM Generation CH1.**
5. **Configure Prescaler and Counter Period for 50 Hz PWM (standard servo frequency):**
   - **System Clock = 48 MHz**
   - **Prescaler = 4799**
   - **Counter Period = 1999**

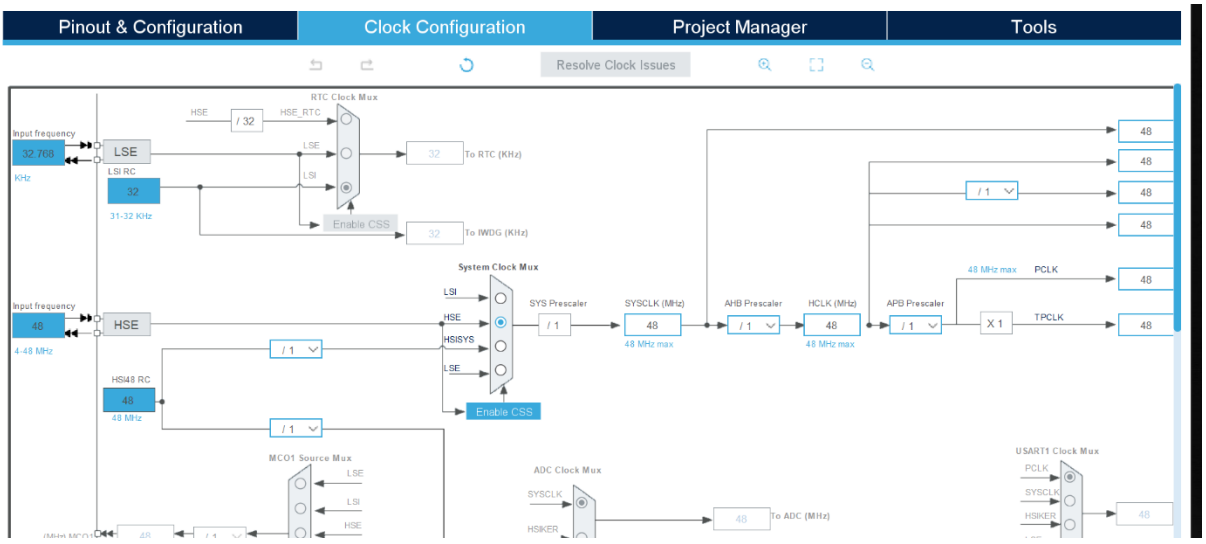
Select Project → Generate Code.

Open project in STM32CubeIDE.

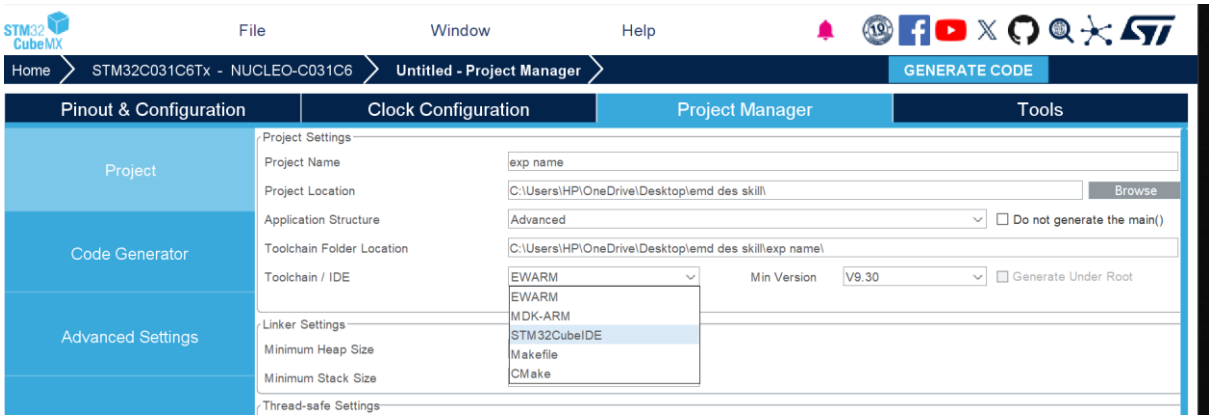








- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened.**
- **Create a main.h header file and paste the program given under main.h**
- **In main .c file copy the given code under main.c**

Output:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **Explain the different types of clocks in NUCLEOC031C6?**

2. **State some practical uses of servo motors?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 7: Interrupts and Event-Driven Programming

Aim/Objective:
Interrupts and Event-Driven Programming

Components Required: STM32 NUCLEOC0316C BOARD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

- **Interrupt: A hardware or software signal that tells the MCU to stop normal execution and run a specific function (ISR — Interrupt Service Routine).**
- **Event-driven programming: Code reacts to events (button press, timer, UART receive) instead of constantly polling.**

Pre-Lab:

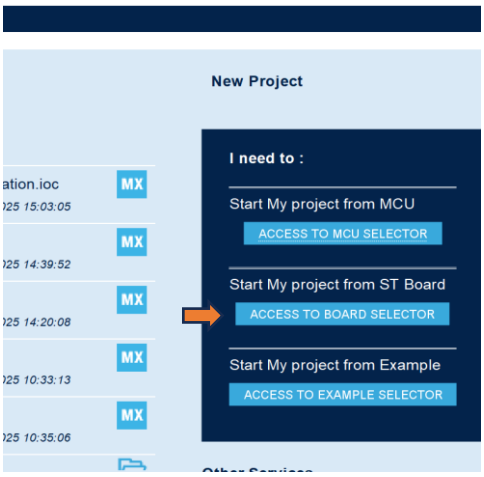
1. **Explain the features of STM32 NUCLEOC031C6?**

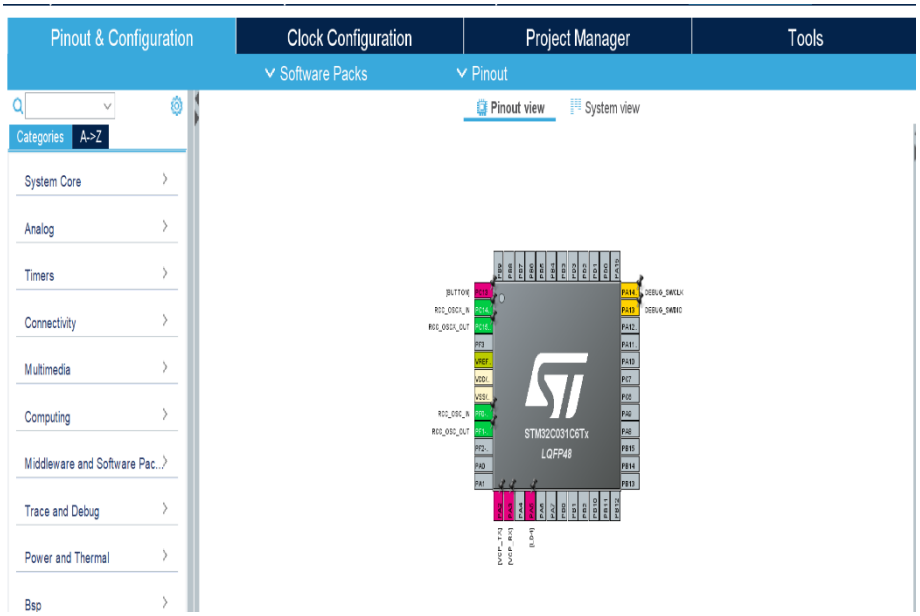
2. **What are interrupts and how they can be handled?**

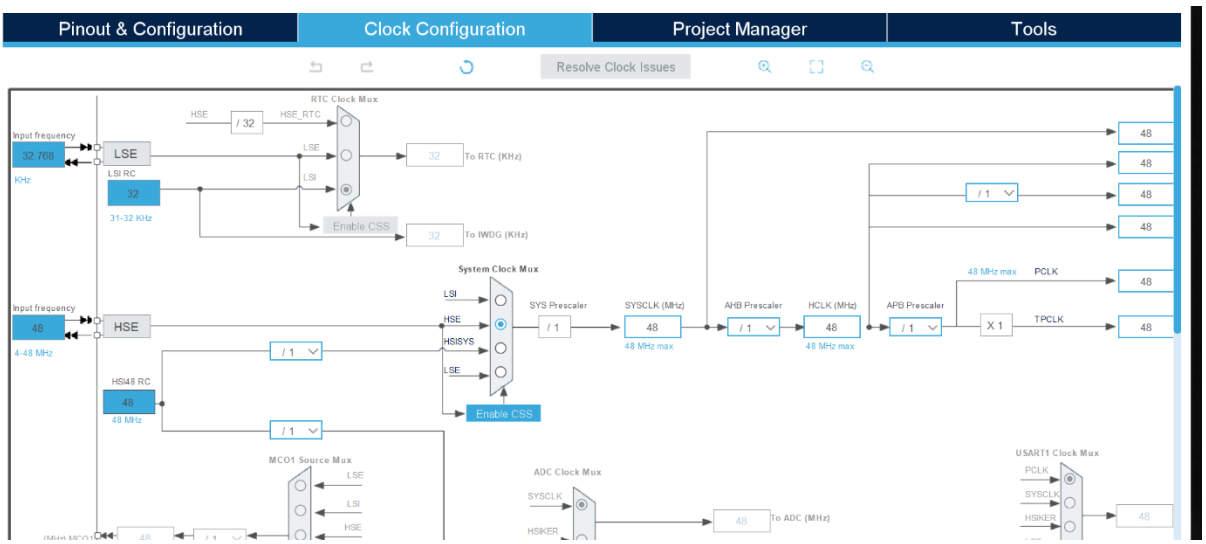
In-Lab:

Procedure:

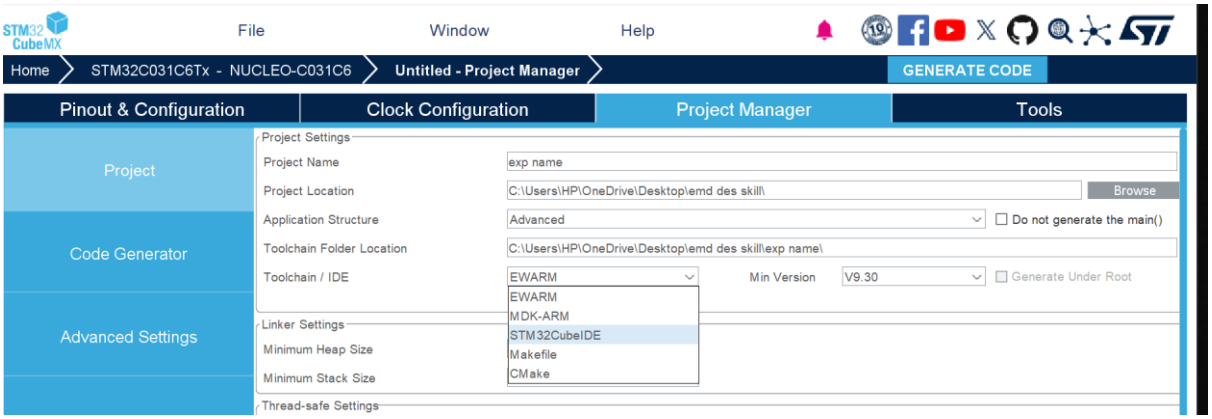
1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**
5. **OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE**







- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **After the successful generation of code just open your main.c file and check for the while loop inside the while loop paste or write the following code to toggle led using pushbutton.**

Output:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **Explain different types of interrupts in an microcontroller u know?**

2. **Write the procedure of your interrupt exp you have performed and their connections ?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 8 : ADC Data Acquisition and Plotting

Aim/Objective:
ADC Data Acquisition and Plotting

Components Required: STM32 NUCLEOC0316C BOARD,16x2 LCD

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**
- **16 X 2 LCD**
- **TEMAPRATURE LM32 OR DHT11 SENSOR**

Description:

A digital thermometer using an STM32 Nucleo board and an LCD display is an embedded system designed to measure temperature and display it digitally in real-time. This project combines a temperature sensor, a microcontroller, and an LCD module to create a compact and accurate temperature monitoring device.

Components:

1. **STM32 Nucleo Board – Acts as the microcontroller unit (MCU) to process temperature readings and control the display.**
2. **Temperature Sensor – Common options include:**
   - **LM35: Analog temperature sensor with output proportional to temperature.**
   - **DS18B20: Digital temperature sensor using the 1-Wire protocol.**
3. **LCD Display – Typically a 16x2 character LCD or a graphical LCD, used to show the measured temperature.**
4. **Connecting Wires & Breadboard – For circuit connections.**

Working Principle:

1. **Temperature Sensing:**
   The sensor detects the ambient temperature and converts it into an electrical signal (analog voltage for LM35 or digital data for DS18B20).
2. **Signal Processing:**
   The Nucleo board reads the sensor output:
   - **For analog sensors, the ADC (Analog-to-Digital Converter) on the STM32 converts voltage into a digital value.**

- **For digital sensors, data is read using a communication protocol like 1-Wire or I2C.**

3. **Temperature Calculation:**
   The MCU calculates the temperature in °C or °F using the sensor's characteristics. For example, LM35 outputs 10 mV per °C, so the MCU converts the ADC value to temperature.

4. **Display Output:**
   The calculated temperature is sent to the LCD. The display is updated periodically (e.g., every second) to show real-time temperature readings.

Pre-Lab:

1. **Explain the uses of an LCD DISPLAY?**

2. **DESCRIBE PINS OF LCD 16X2?**

In-Lab: System Clock and Power Modes

Procedure:

1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**

CONNECTIONS

LCD Pin Nucleo C031C6

SDA D15 → PB9

LCD Pin Nucleo C031C6

SCL D14 → PB8

VCC 5V

GND GND

DHT11 VCC → 5V

DHT11 GND → GND

DHT11 DATA → A1 (PA1) with pull-up 4.7kΩ to VCC

Step 1: Create New Project

1. **Open STM32CubeMX.**
2. **Click New Project → Select STM32C031C6 → Start Project.**
3. **Give a name (e.g., DHT11_UART).**

Step 2: Configure GPIO for DHT11

1. **Go to Pinout & Configuration.**
2. **Locate PA1 (we will use this for DHT11 DATA).**
3. **Click on PA1 → Set GPIO_Input.**
4. **In GPIO Settings:**
   - **Mode: Input**

Go to Clock Configuration tab.

Make sure HSI is selected as System Clock Source.

Set SYSCLK = 48 MHz.

APB1 = 48 MHz (default for C031C6).

Click Apply.

Step 3: Configure TIM3 for Microsecond Delay

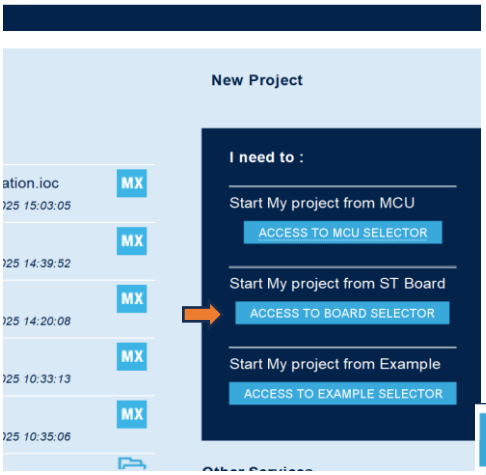
1. **Go to Timers → TIM3.**

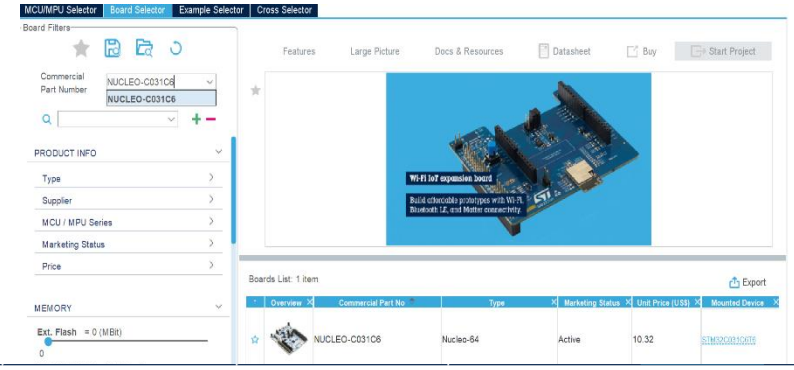
2. **Mode: Base.**
3. **Prescaler: 48-1 → this gives 1 MHz counter (1 tick = 1 µs if clock = 48 MHz).**
4. **Counter Mode: Up.**
5. **Period: 65535.**
6. **Clock Division: DIV1.**
7. **Click Apply.**

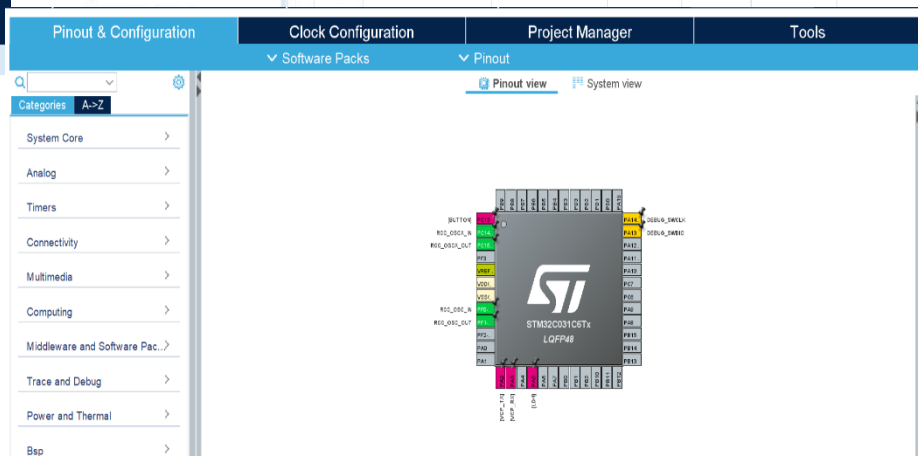
   Go to Connectivity → USART1.

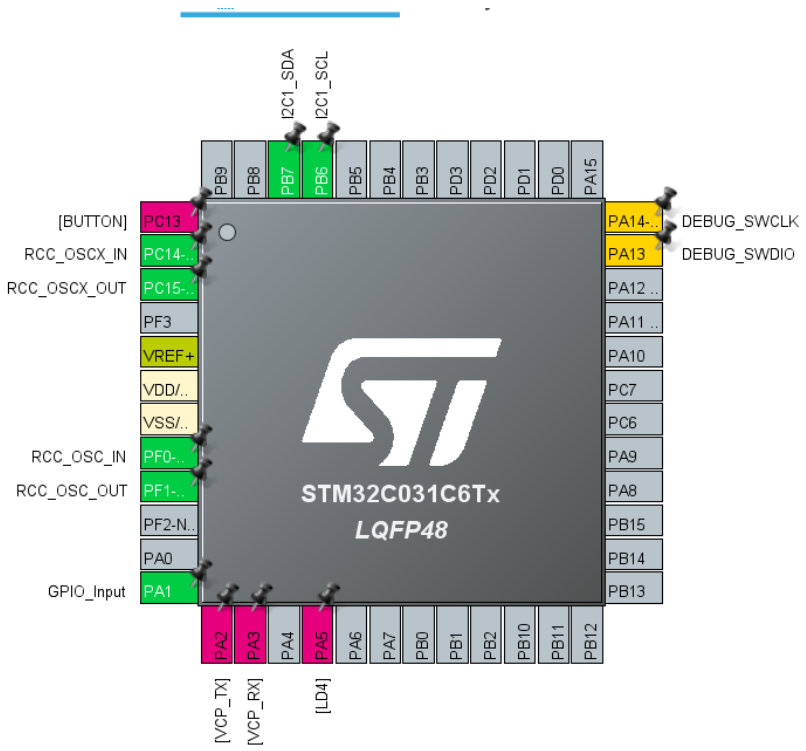
   Mode: Asynchronous.

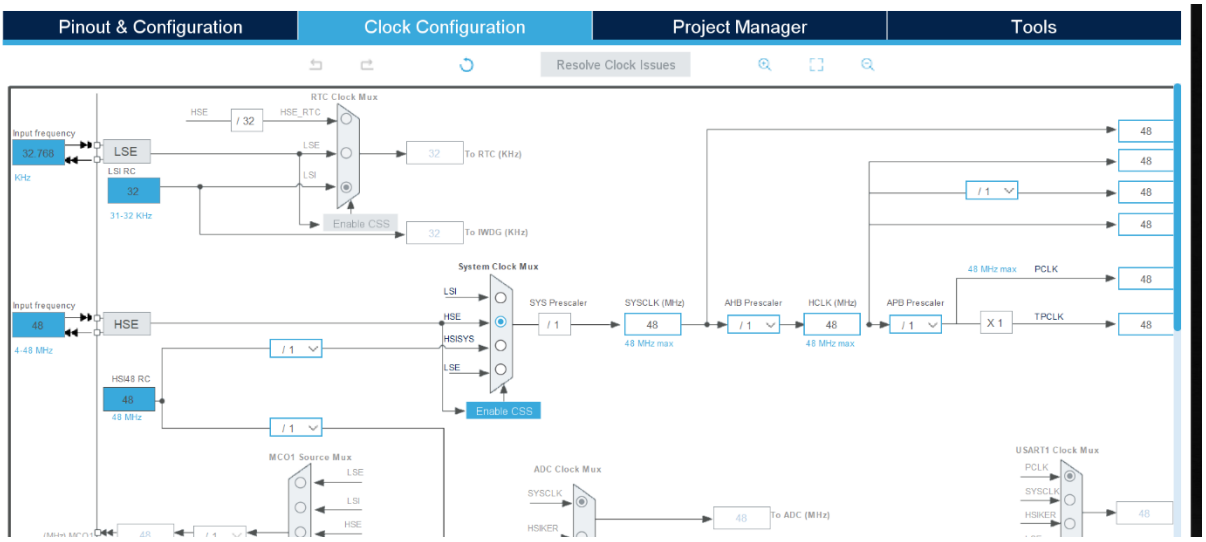
OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE



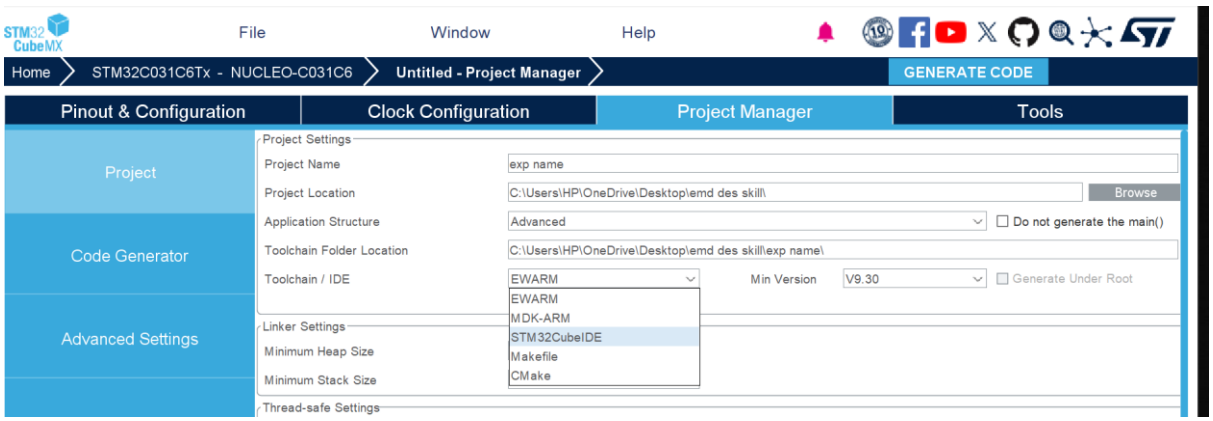








- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **Select file->Core->Src->main and copy the given program and paste**

Plotting on PC

Option 1: Arduino Serial Plotter

- **Open Serial Monitor**
- **Select correct COM port**
- **Baud rate 115200**
- **Click Serial Plotter**

Option 2: Python (Matplotlib)

import serial

```
import matplotlib.pyplot as plt
ser = serial.Serial('COM3', 115200)
data = []
plt.ion()
while True:
    value = int(ser.readline().decode().strip())
    data.append(value)
    if len(data) > 100:
        data.pop(0)
    plt.clf()
    plt.plot(data)
    plt.pause(0.01)
```

Output

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **What is an data acquisition and explain a type of it?**

2. **What are the features of NUCLEOC031C6?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 9 : UART Serial Communication

Aim/Objective:
UART Serial Communication

Components Required: STM32 NUCLEOC0316C BOARD,

Pre-Requisites:

- **STM32 CUBEMX**
- **STM32 CUBEIDE**

Description:

This project demonstrates UART serial communication using FreeRTOS on the STM32 NUCLEO-C031C6 microcontroller. The objective is to show how real-time tasks and interrupts can be used together to achieve reliable and non-blocking serial communication in an embedded system.

The system uses two FreeRTOS tasks: one task is responsible for periodically transmitting data over UART, while another task handles receiving data. UART reception is interrupt-driven to avoid blocking the CPU. Received data is safely transferred from the interrupt service routine to a task using a FreeRTOS queue, ensuring proper synchronization between ISR and tasks.

Components:

- **STM32 NUCLEOC031C6**

Pre-Lab:

1. **Explain the UART COMMUNICATION PROTOCAL?**

2. **EXPLAIN OTHER COMMUNICATION PROTOCOLS YOU KNOW?**

In-Lab:

Procedure:

1. **Open CubeMX and select ACCESS TO BOARD SELECTION**
2. **SELECT BOARD NAME NUCLEOC0316C**
3. **CLICK ON THE BOARD IMAGE ON RIGHT SIDE**
4. **AFTER RENDERING UI SUCCESFULLY**

CONNECTIONS

USART

PA2 -> TX

PA3 -> RX

LED -> PD12

Step 1: Create New Project

1. **Open STM32CubeMX.**
2. **Click New Project → Select STM32C031C6 → Start Project.**
3. **Give a name (e.g., Commandline_UART).**

Step 2: Serial Terminal Setup (PC)

- **Tool: PuTTY / TeraTerm / Arduino Serial Monitor**
- **Baud rate: 115200**
- **Data bits: 8**
- **Parity: None**
- **Stop bits: 1**
- **Line ending: Newline (CR/LF)**

Go to Clock Configuration tab.

Make sure HSI is selected as System Clock Source.

Set SYSCLK = 48 MHz.

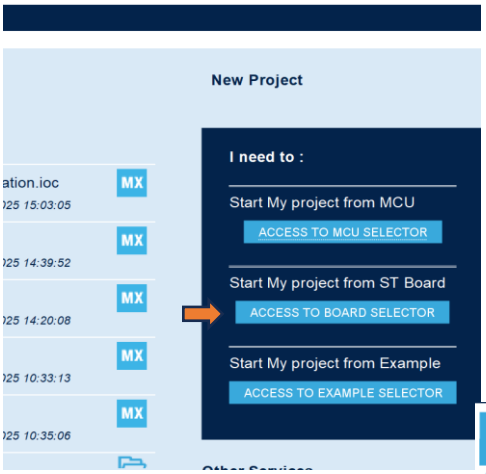
APB1 = 48 MHz (default for C031C6).

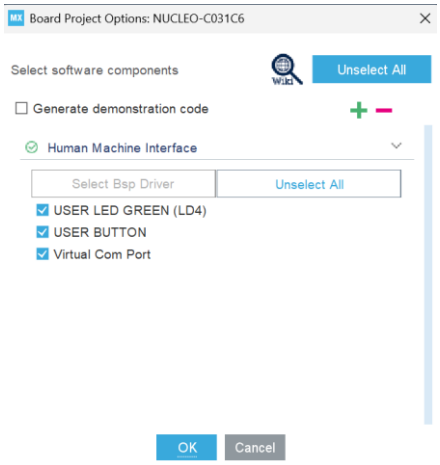
Click Apply.

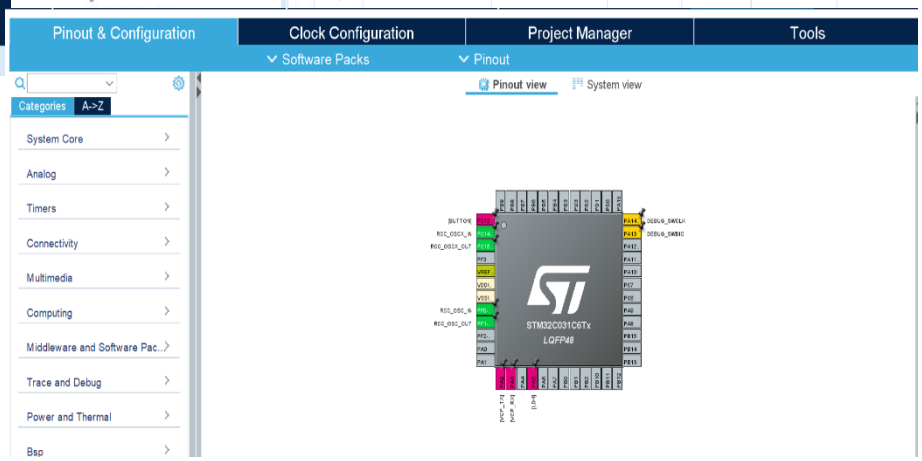
Step 3: Go to Connectivity → USART2.

Mode: Asynchronous.

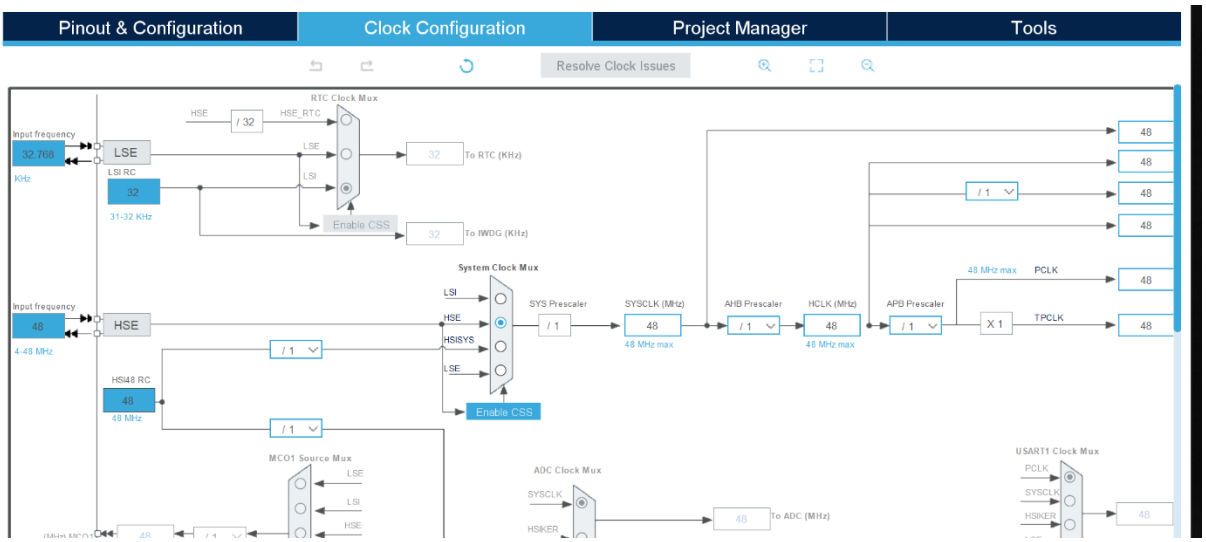
OPEN PROJECT MANAGER ->GIVE YOUR FILE NAME ->SELECT STM32CUBEIDE IN PLACE OF EWARM AND GENERATE THE CODE AND WAIT UNTIL PROJECT OPENS IN CUBE IDE BY CLCIKING OPEN PROJECT IN STMCUBE IDE



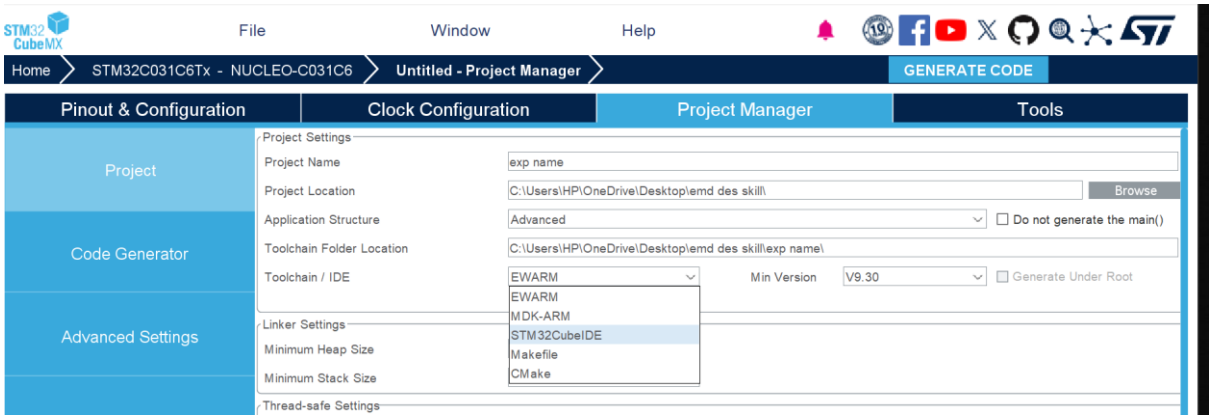




Give the appropriate connections to the board



- **SELECT THE CLOCK FREQUENCY TO 48MHZ IN HSE**



- **Add your file name or project name-> IDE STM32CUBEIDE->GENERATE CODE**
- **After clicking on Open project**
- **CubeIDE is opened**
- **Select file->Core->Src->main and copy the given program and paste**

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **HOW DO WE INITIALISE UART COMMUNICATION BETWEEN 2 DEVICES?**

2. **What are the features of NUCLEOC031C6?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 10 : RTOS Task Scheduling

Aim/Objective:
RTOS Task Scheduling

Components: QNX Software Center, QNX Momentics IDE, VMWARE Workstation

Description:

RTOS task scheduling is the mechanism by which the Real-Time Operating System decides which task should run on the CPU at any given time. In an RTOS, multiple tasks appear to run simultaneously, but the scheduler rapidly switches the CPU between tasks based on priority and timing constraints.

Pre-Lab:

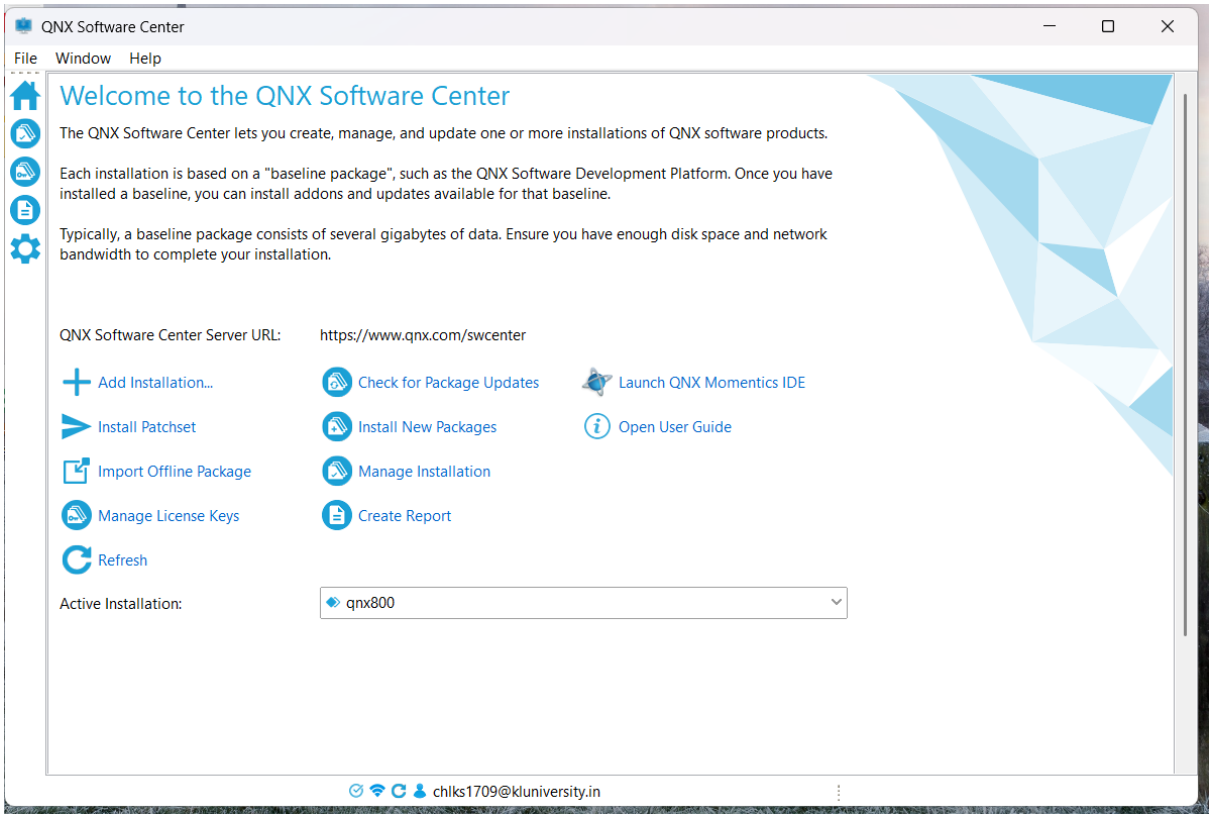
1. **WHAT IS AN REAL TIME OPERATING SYSTEMS ?**

2. **DIFFERENTIATE HARD AND SOFT REAL TIME SYSTEMS?**

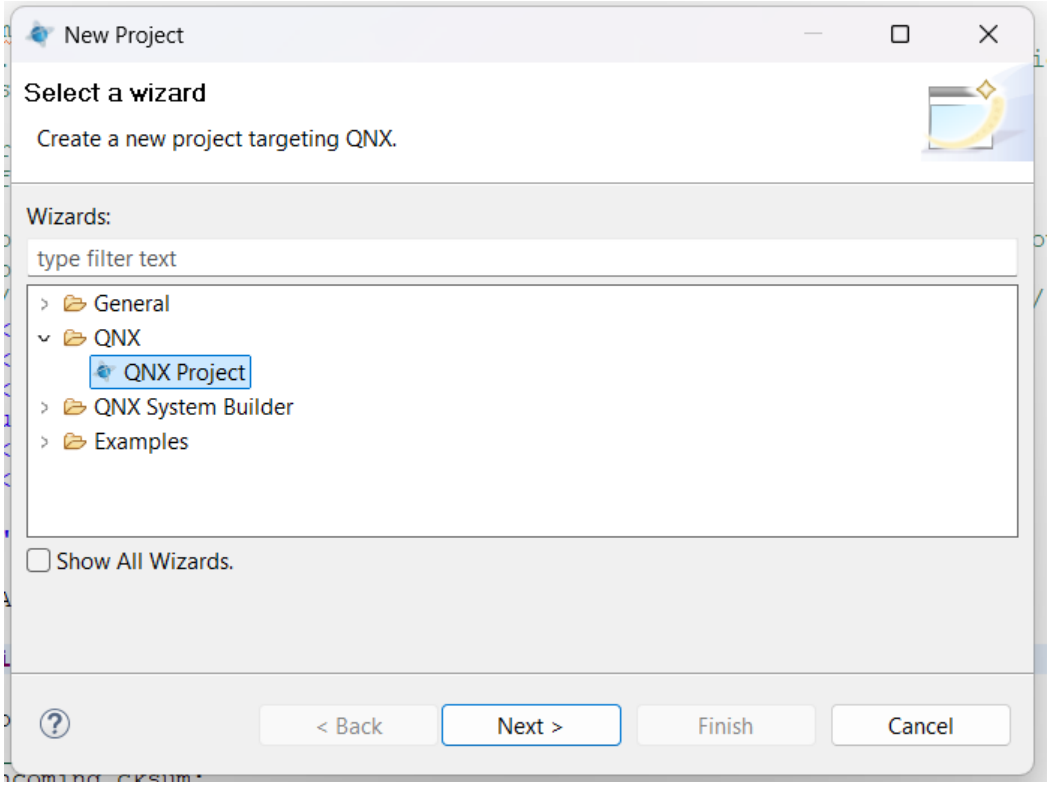
In-Lab: RTOS Task Scheduling

Procedure:

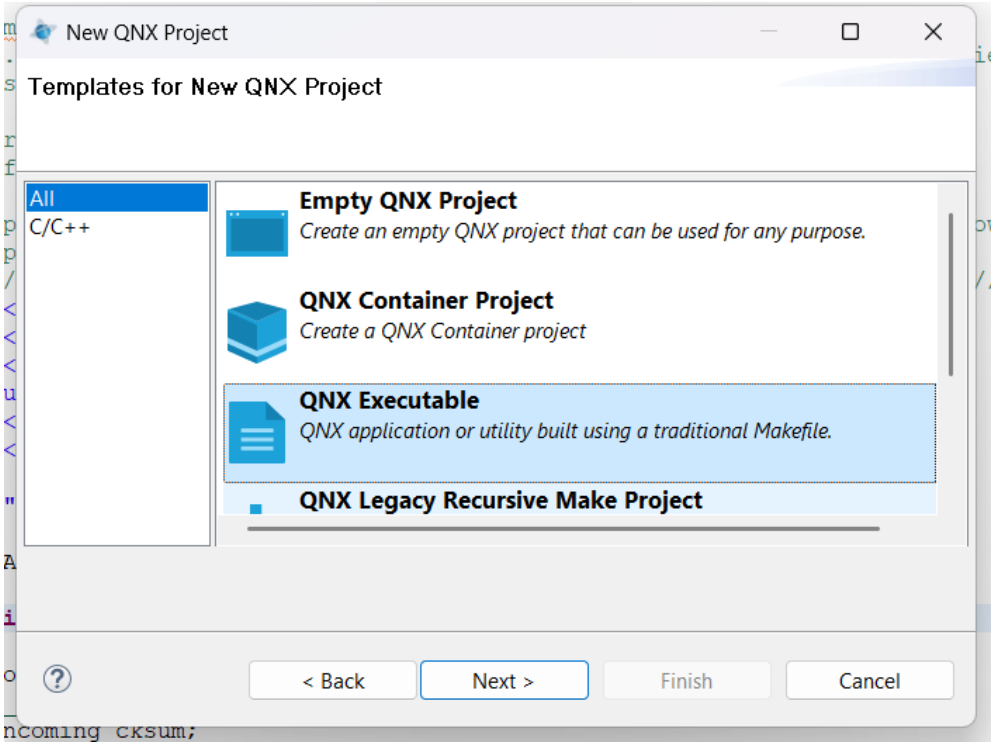
- **Open VMWARE Workstation and switch on virtual machine (QNX).**
- **Open QNX Software Center and click on Launch QNX Momentics IDE and make sure that qnx800 is installed and selected in QNX Software Center.**



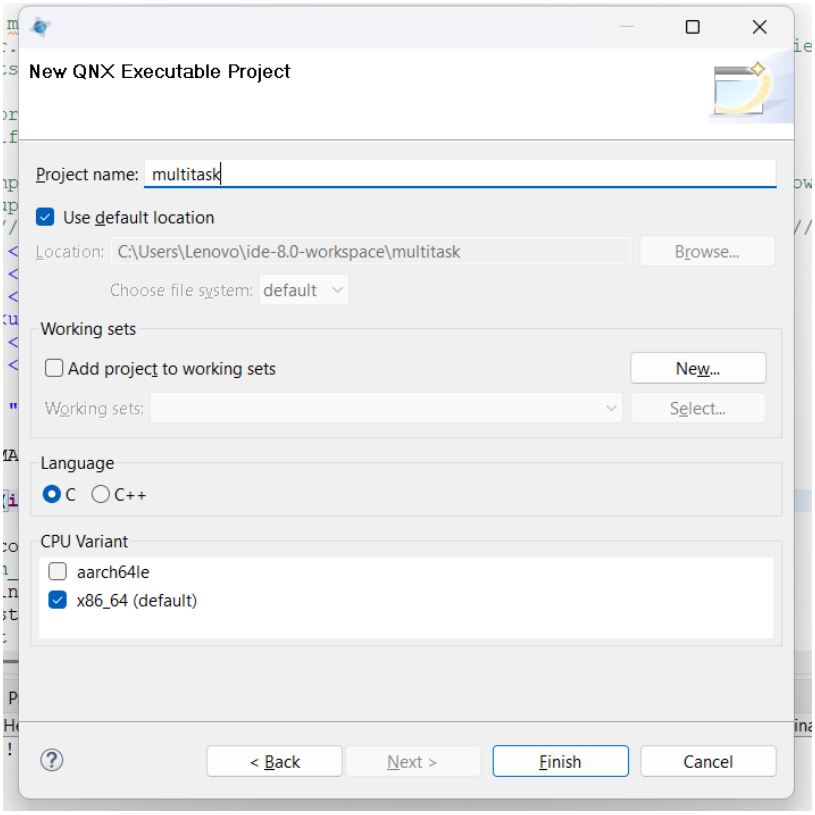
- **Go to File menu and click on New -> Project**
- **Now click on QNX project in the displayed window.**



Click on Next and select QNX Executable and click on next.



- **Now type the project name and select CPU Variant as x86_64 (default) and click on finish.**



- **It will create a Hello-world project. Delete the current program and type your code.**
- **For execution**
  - **Right click on the project name in the project pane and select clean project.**
  - **Then right click on the project name in the project pane and select build project.**
  - **It will create a Binaries folder. Open the binaries folder, and right-click on the file and select Run As -> C/C++ QNX Application**

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **WHAT IS AN HARD REAL TIM E SYSTEM AND EXAMPLE?**

2. **What are the features of NUCLEOC031C6?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 11 : Inter-Task Communication (Queues/Semaphores)

Aim/Objective:
Inter-Task Communication (Queues/Semaphores)

Components Required: QNX Software Center, QNX Momentics IDE, VMWARE Workstation

Description:

Inter-Task Communication (ITC) is a concept in RTOS (Real-Time Operating Systems) that allows multiple tasks to safely share data and synchronize their actions while running concurrently.

In simple words:
It is how tasks "talk to each other" and "coordinate" without causing errors.

Pre-Lab:

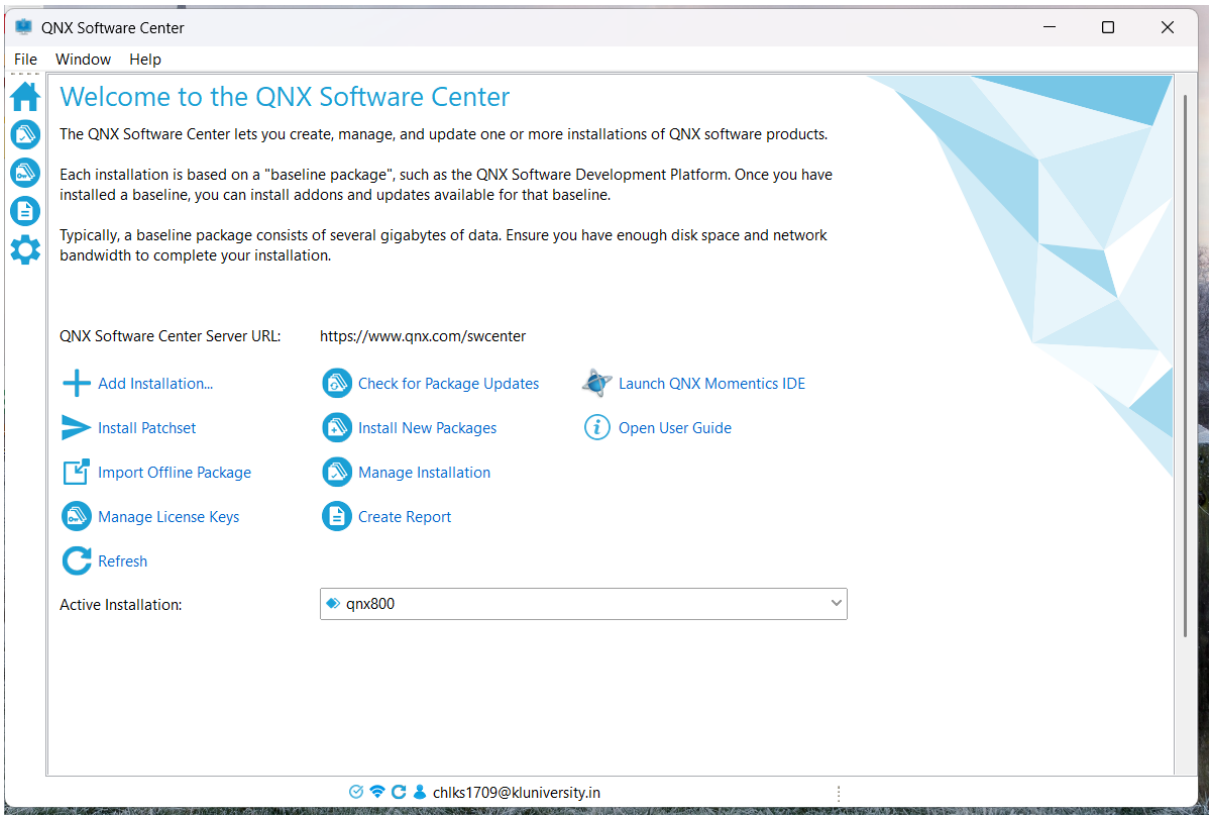
1. **WHAT IS AN Inter-Task Communication (ITC)**

2. **DIFFERENTIATE HARD AND SOFT REAL TIME SYSTEMS?**

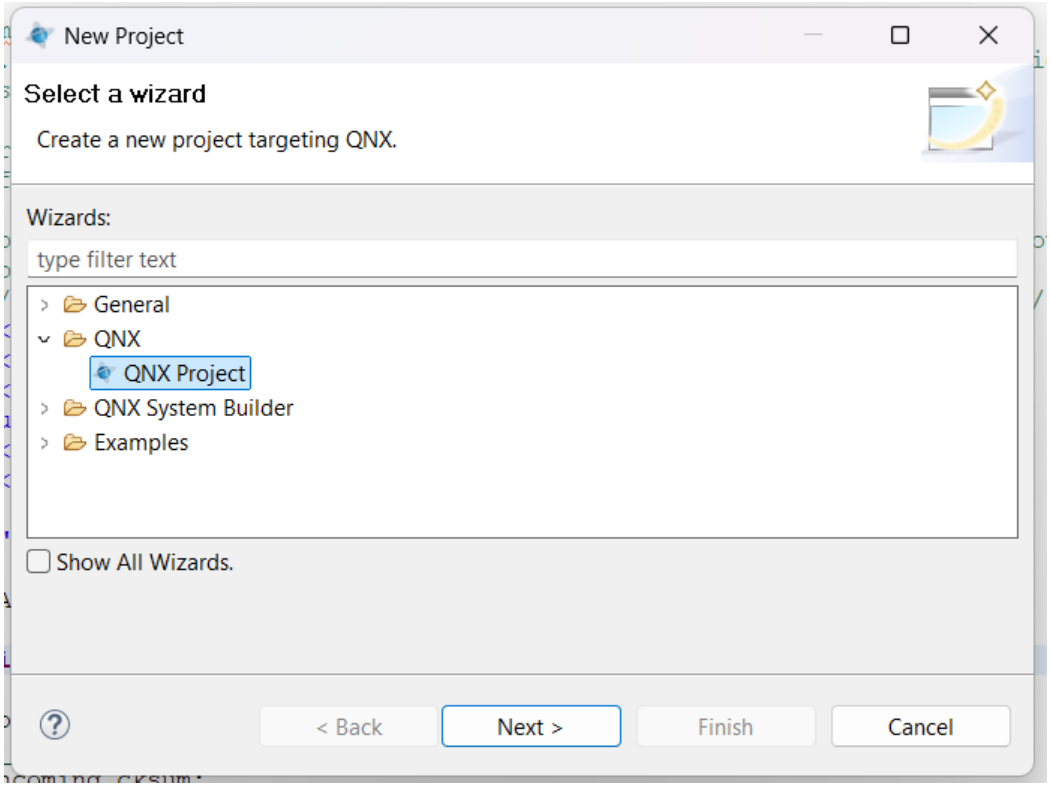
In-Lab: Inter-Task Communication

Procedure:

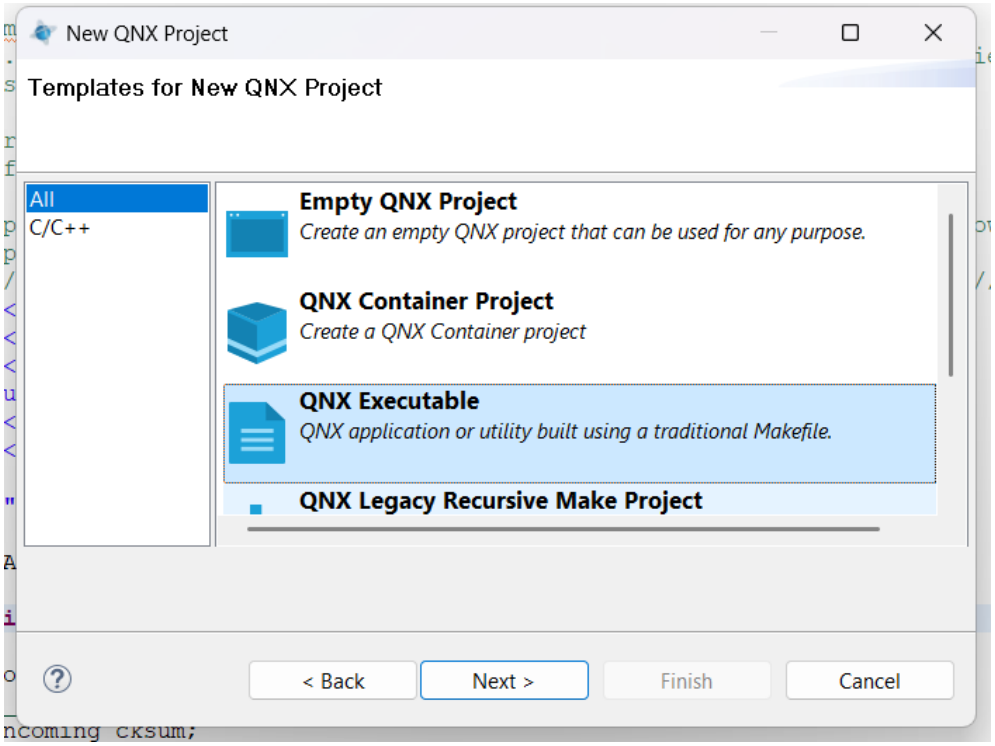
- **Open VMWARE Workstation and switch on virtual machine (QNX).**
- **Open QNX Software Center and click on Launch QNX Momentics IDE and make sure that qnx800 is installed and selected in QNX Software Center.**



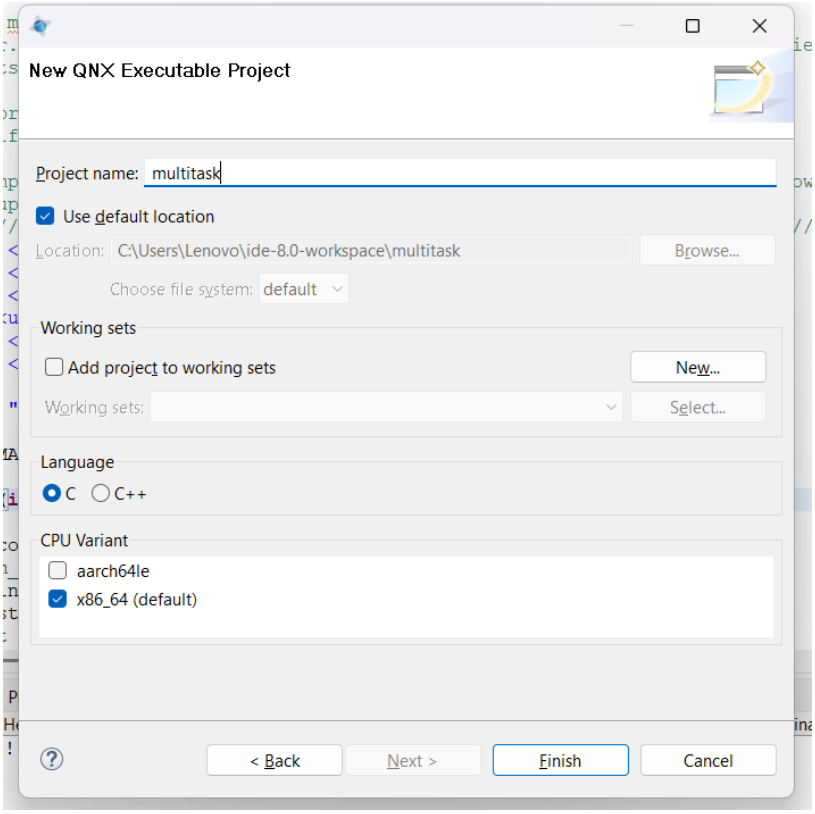
- **Go to File menu and click on New -> Project**
- **Now click on QNX project in the displayed window.**



- **Click on Next and select QNX Executable and click on next.**



- **Now type the project name and select CPU Variant as x86_64 (default) and click on finish.**



- **It will create a Hello-world project. Delete the current program and type consumer.c code.**
- **For execution**
  - **Right click on the project name in the project pane and select clean project.**
  - **Then right click on the project name in the project pane and select build project.**
  - **It will create a Binaries folder. Open the binaries folder, and right-click on the file and select Run As -> C/C++ QNX Application**

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **WHAT IS AN REAL TIM E SYSTEM AND EXAMPLE ?**

2. **What IS DIFFERENCE BETWEEN PROCESSOR AND CONTROLLER?**

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 12 : Timer-Based Real-Time Applications

Aim/Objective:

Timer-Based Real-Time Applications

Components Required: QNX Software Center, QNX Momentics IDE, VMWARE Workstation

Description:

Timer-based real-time applications are systems where tasks are executed at specific time intervals or after a precise delay using hardware or software timers. These applications rely on accurate timing to function correctly.

Pre-Lab:

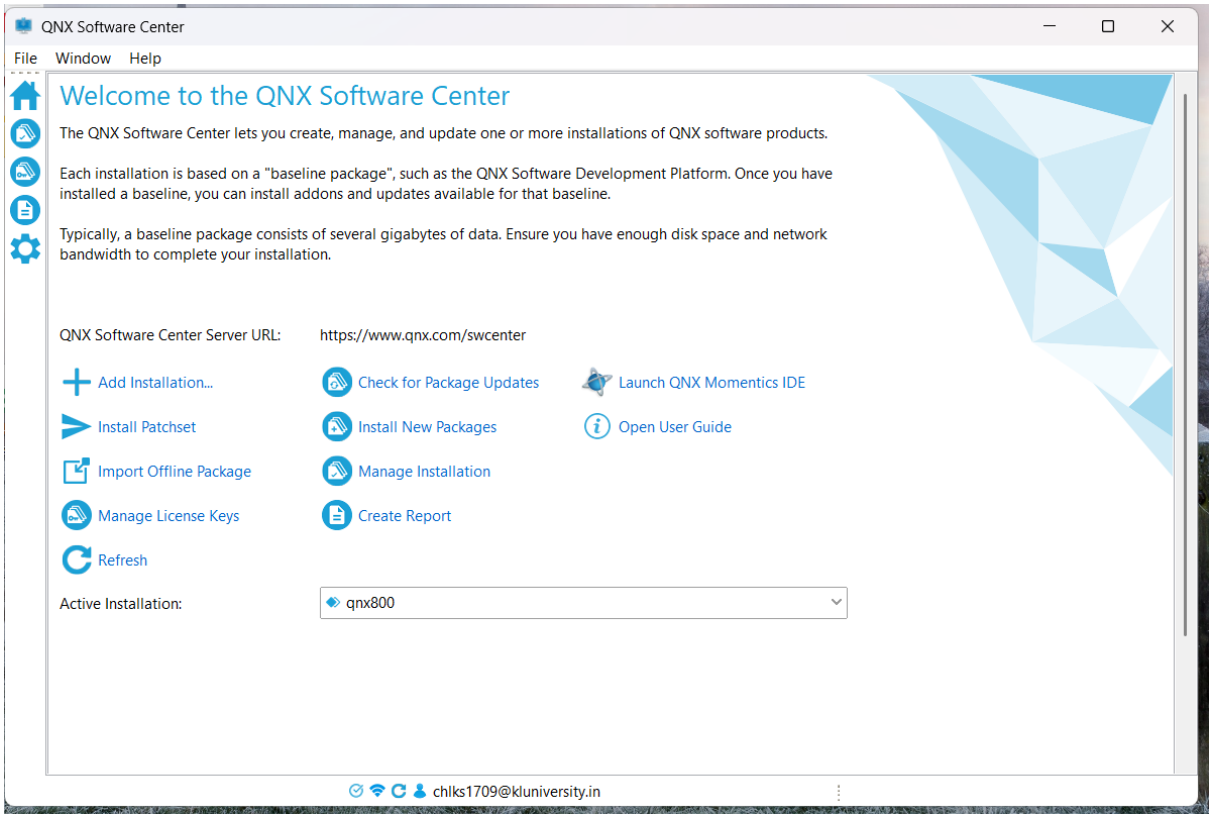
1. **WHY TIMING IS AN IMPORTANT FACTOR IN RTOS?**

2. **PROVIDE FEW EXAMPLES OF RTOS IN REAL LIFE ?**

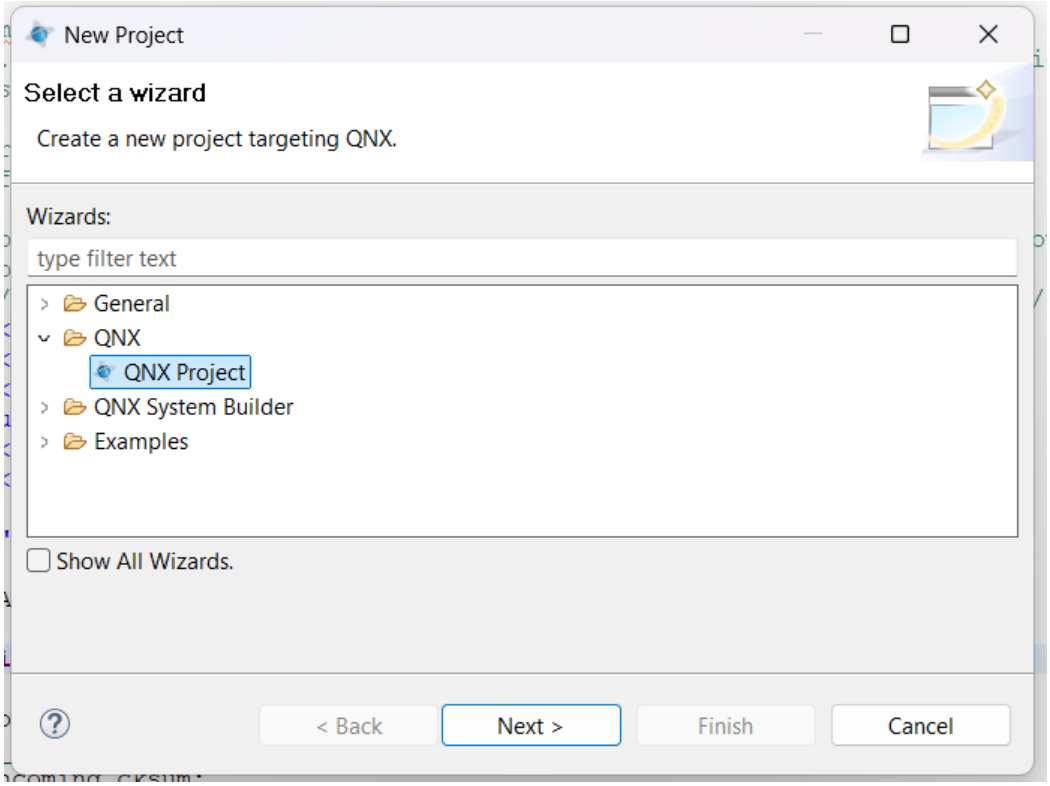
In-Lab: Implement a real-time clock with periodic display update and event alarms.

Procedure:

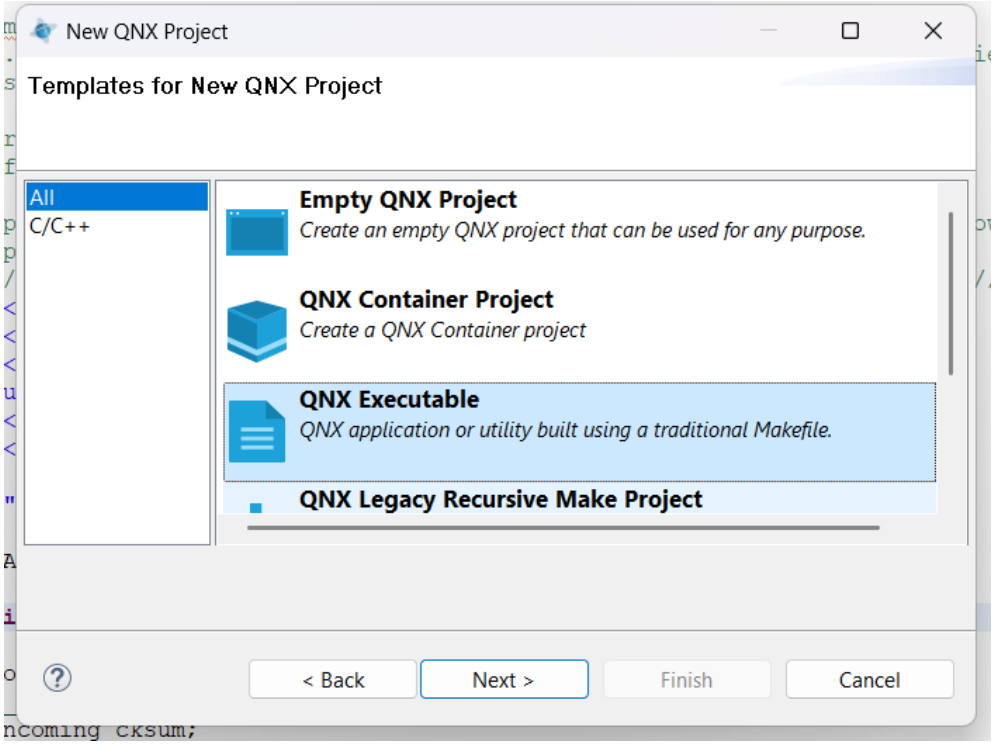
- **Open VMWARE Workstation and switch on virtual machine (QNX).**
- **Open QNX Software Center and click on Launch QNX Momentics IDE and make sure that qnx800 is installed and selected in QNX Software Center.**



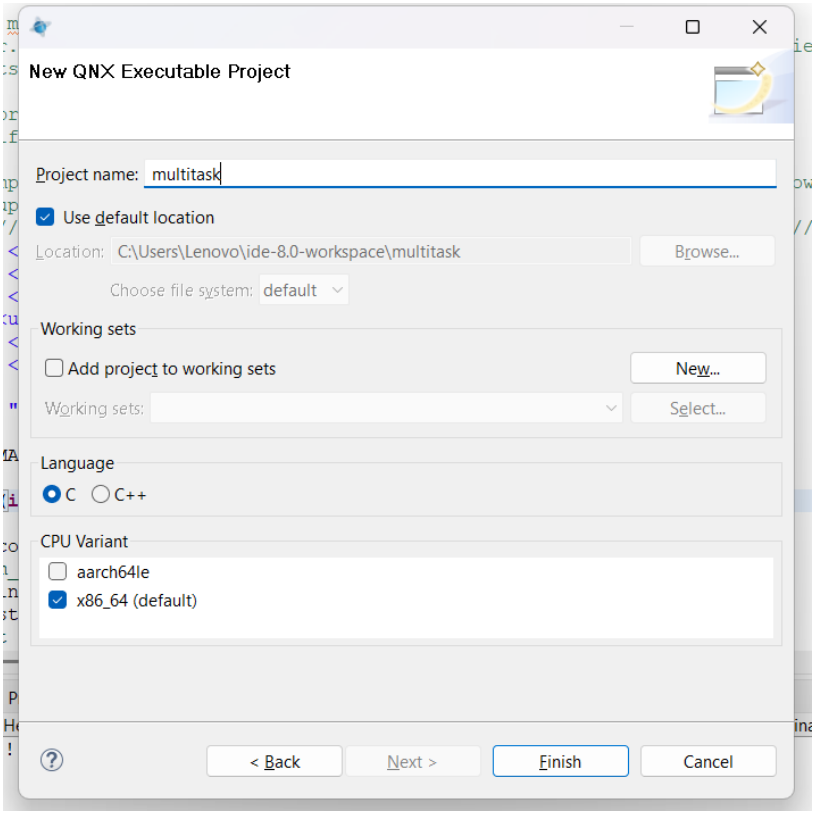
- **Go to File menu and click on New -> Project**
- **Now click on QNX project in the displayed window.**



- **Click on Next and select QNX Executable and click on next.**



- **Now type the project name and select CPU Variant as x86_64 (default) and click on finish.**



- **It will create a Hello-world project. Delete the current program and type your code.**
- **For execution**
  - **Right click on the project name in the project pane and select clean project.**
  - **Then right click on the project name in the project pane and select build project.**
  - **It will create a Binaries folder. Open the binaries folder, and right-click on the file and select Run As -> C/C++ QNX Application**

OUTPUT:

Data and Results:

Analysis and Inferences:

Post Lab:

1. **Paste the Screen Shots of the outputs for the experiment.**
2. **Write Results, Inference Analysis.**

VIVA-VOCE Questions:

1. **EXPLAIN THE KEY FEATURES OF NUCLEOCO31C6?**

2. WRITE THE STEPS INVOVLED IN CREATING A PROJECT IN STMCUBE MX AND IDE?

3.EXPLAIN REAL TIME OPERATING SYSTEMS?

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date

Experiment Title 13: Project

Aim/Objective:

Create a Project using a Microcontroller that supports Real Time Tasks.

Data and Results:

Analysis and Inferences:

Evaluator Remark (if Any):	**Marks Secured ___ out of 50**
	Signature of the Evaluator with Date